

Danmarks
Tekniske
Universitet



34249 - Malware Detection Final Report

AUTHORS

Laurits M. Jensen - s194279

June 24, 2026

Henrik Wessing
DTU ELECTRO
Institut for Elektroteknologi og Fotonik
Networks Technology and Service Platforms
Ørstedes Plads
Bygning 343, Rum 228
2800 Kgs. Lyngby
Denmark

Att. Lars Staalhagen

May 31, 2026
34249 Midterm Report
Laurits M. Jensen

Dear Henrik Wessing,

Please find enclosed the midterm report for the project entitled *Malware Detection*, submitted as part of the requirements for 34249 Project Work.

The report presents the current status of the project and documents the development of a Docker-based prototype for behavior-based malware analysis. It describes the project background, implementation, interfaces, testing activities, discussion of the scope refinement, and the remaining work toward final delivery.

The current prototype focuses on isolated execution, behavioral observation, and verdict generation for suspicious files. While the original project scope also included a broader orchestration perspective involving Kubernetes, the implemented work at this stage is centered on the malware-analysis core.

Respectfully submitted,

Abstract

This report presents a Docker-based behavior-based malware-analysis system for detecting suspicious files through isolated execution and behavioral analysis. The system implements a hybrid approach combining rule-based detection with AI-driven assessment using local large language model (LLM) inference via Ollama.

Traditional signature-based antivirus methods are effective against known threats but fail against zero-day exploits and polymorphic malware. Behavior-based analysis addresses this by observing what programs *do* rather than what they *are*. The implemented system executes suspicious files in isolated Docker containers and monitors the resulting behavioral artifacts: filesystem modifications, process execution patterns, network communications, and persistence mechanisms. These artifacts are then evaluated by both a rule-based verdict system and an AI agent to determine maliciousness.

The architecture comprises a Python control layer managing isolated container execution, comprehensive behavioral monitoring through filesystem snapshots and network inspection, and a dual-verdict pipeline combining explicit rule-based classification with contextual reasoning via LLM-based analysis. The AI agent leverages transformer-based reasoning to understand behavioral patterns that traditional rule-based systems struggle to contextualize, such as the combination of file encryption, deletion, and network silence indicating ransomware behavior.

Testing demonstrates that the prototype successfully identifies file-system tampering, executable payload creation, privilege escalation attempts, and persistence mechanisms. The hybrid approach provides both the transparency of rule-based verdicts and the contextual reasoning of AI-driven assessment, with local LLM inference preserving privacy by ensuring sensitive execution logs never leave the analysis environment. This work establishes a foundation for behavior-based malware detection leveraging modern AI techniques for improved detection accuracy and contextual understanding.

Contents

List of Figures	vi
List of Tables	vi
Acronyms	vii
Glossary	viii
1 Introduction	1
2 Project Overview	3
2.1 Project Plan	3
2.1.1 Project Summary	3
2.1.2 Top-Level Work Packages	3
2.1.3 Timeplan	3
2.1.4 Tasks	6
2.2 System Overview	8
2.2.1 Main Functionality	8
2.2.2 System Architecture and Subblocks	9
2.2.3 Scope and Requirements	11
3 Theory and Background	12
3.1 Behavior- vs Signature-Based Malware Detection	12
3.2 Static- vs Dynamic Analysis	12
3.3 Docker and Containers	13
3.3.1 Docker	13
3.4 Docker Networking	14
3.4.1 Network Container Archetypes	14
3.4.2 Docker Network Drivers	15
3.5 Container-Based Malware Analysis	16
3.6 Artificial Intelligence in Cybersecurity and Malware Detection	17
3.7 AI Agents for Behavioral Assessment	18
3.8 Trust, Explainability, and the Black Box Problem	19
4 Implementation	21
4.1 Implementation Strategy	21
4.2 Docker-Based Environment	21
4.3 Python Control Layer	23
4.3.1 Execution Pipeline	23
4.3.2 Kubernetes Substitute	24
4.4 Scanning and Execution Workflow	24
4.5 Verdict Logic	24
4.5.1 Rule-Based Verdict (Legacy Approach)	24

4.5.2	AI Agent-Based Verdict (Current Approach)	25
4.6	Malware Behavior Focus	25
4.7	Ollama Wrapper Integration	26
4.8	Summary	26
5	Interfaces	27
5.1	Overview of the Implemented Interfaces	27
5.1.1	Interface Workflow	28
5.2	User Interface to the Python Control Layer	28
5.3	Python Control Layer to Docker Runtime	29
5.4	Scanner Interface to the Sandbox Environment	29
5.5	Filesystem and Artifact Interface	30
5.6	Runtime Monitoring Interface	31
5.7	Analysis Output Interface	31
5.8	Execution Log Interface	32
5.9	AI Analysis Interface	32
5.10	Assumptions and Interface Constraints	33
5.11	Summary	33
6	Tests and Early Results	34
6.1	Test Setup	34
6.2	Test Scenarios	34
6.3	Early-stage Testing	35
6.3.1	Destructive Script Simulation	35
6.3.2	Ransomware Simulation	35
6.3.3	Agent Installation Simulation	36
6.4	Testing and Calibrating the Verdict	37
6.4.1	Traffic Verdict Testing	37
6.4.2	Agent Installation with Traffic Verdict	38
6.5	Testing Full Malware Program: RedTail	39
6.6	Results of the Previous Rule-Based Approach	40
6.7	Limitations of the Previous Rule-Based Verdict	40
6.8	Why the System Transitioned to an AI Agent	40
6.9	AI Agent-Based Verdict Testing	41
6.9.1	Selected models for testing	41
6.9.2	Model comparison	41
6.9.3	Runtime comparison	42
6.9.4	Reference Severity for the Test Cases	42
6.9.5	Discussion of Verdict Quality	43
6.9.6	Cross-Model Observations	43
6.9.7	Severity Calibration versus Classification Accuracy	44
6.9.8	Runtime and Practical Trade-offs	44
6.9.9	Model Selection	44
6.9.10	Threats to Validity	45

7	Discussion	46
7.1	Scope Refinement	46
7.2	Tradeoffs	46
7.3	Limitations	46
7.4	Future Perspective	47
8	Conclusion	48
	References	50

List of Figures

1	Project timeplan and development timeline	4
2	Timeplan zoom (26/04/2026 - 26/06/2026)	5
3	High-level system overview	8
4	Conceptual architecture of the malware analysis platform	9
5	Four container network archetypes and their interaction with the Docker network topology [1]	14
6	Linux isolation mechanisms used to construct Docker containers [2]	16
7	Overview of the Docker-based environment and the interaction between the host, scanner, sandbox, and Ollama containers.	22
8	Simplified workflow showing module interaction and data flow through the execution pipeline. helpers.py acts as the central integration point between Python application logic and the Docker runtime.	23
9	Interface Workflow	28
10	Detection of malicious file evil.sh	35
11	Detection of malicious file ransomware.sh	35
12	Detection of malicious file agent.sh	36
13	Testing the traffic verdict	37
14	Agent Installer with Traffic Verdict	38
15	RedTail Cryptominer collected on HoneyPot	39

List of Tables

1	Summary information on the timeplan table (see Figure 1	3
2	Top-level WBS items and major report milestone	3
3	Scope pivot timeline	3
4	Task list extracted from the PDF	7
5	Comparison of severity levels and risk scores produced by the tested Ollama models.	41
6	Runtime comparison for the reported models across the five AI-verdict test cases.	42
7	Reference severity expectations used to assess model calibration, with the primary behavioral indicator motivating each expectation.	42
8	Artificial intelligence tools used during the project.	49

Acronyms

API Application Programming Interface.

C2 Command and Control.

CLI Command-Line Interface.

CPU Central Processing Unit.

HTTP Hypertext Transfer Protocol.

I/O Input/Output.

IP Internet Protocol.

PID Process Identifier.

POSIX Portable Operating System Interface.

RAM Random Access Memory.

SHA-256 Secure Hash Algorithm 256-bit.

SSH Secure Shell.

TCP Transmission Control Protocol.

URL Uniform Resource Locator.

WBS Work Breakdown Structure.

WSL Windows Subsystem for Linux.

YAML YAML Ain't Markup Language.

ZIP ZIP Archive Format.

Glossary

.bin A generic binary executable file.

.elf An executable file in the Executable and Linkable Format used on Unix-like systems.

.py A Python script file.

.run An executable script or installer file commonly used on Linux systems.

.sh A shell script file.

AbuseIPDB An IP reputation service used to look up abuse reports and confidence scores for suspicious IP addresses.

containerd A container runtime used to manage the lifecycle of containers.

Cowrie An SSH and Telnet honeypot used to capture attacker activity and collect malware samples.

honeypot A deliberately exposed or monitored system used to attract attackers and collect information about their behavior.

MalwareBazaar A public malware sample sharing platform used to obtain malware archives and related metadata.

payload The part of a malicious program that performs the intended harmful action after execution.

sandbox An isolated execution environment used to run suspicious files safely while observing their behavior.

Tor The Onion Router, a network designed to anonymize internet traffic by routing it through multiple relays.

1 Introduction

Problem Statement

Contemporary cybersecurity faces a fundamental challenge: malware continues to evolve faster than signature-based detection can adapt. Each year, millions of new malware variants are discovered, many of which represent minor modifications of known families specifically designed to evade traditional antivirus signatures [3]. This cat-and-mouse dynamic creates a persistent gap between threat development and defensive capabilities.

Behavioral analysis offers a principled alternative to signature-based detection. Rather than identifying threats through predefined patterns, behavioral analysis evaluates programs based on their actions during execution. The insight is that malicious intent manifests through observable actions—malware cannot hide what it does, only what it is. By observing execution in a controlled environment, it becomes possible to detect suspicious behavior regardless of the malware’s structural modifications.

Project Motivation and Objectives

This project explores behavior-based malware detection through containerized execution and behavioral analysis, with a focus on understanding how modern AI techniques can enhance detection accuracy and contextual reasoning in security applications.

The primary objectives are:

1. Implement a Docker-based sandbox environment for safe, isolated malware execution
2. Develop behavioral monitoring to capture filesystem, process, and network activity
3. Investigate both rule-based and AI-driven assessment approaches for malware classification
4. Evaluate the effectiveness of hybrid detection strategies combining explicit rules with learned contextual reasoning

This work contributes to the broader field by demonstrating how local, privacy-preserving AI inference (via Ollama) can complement traditional detection methods, and by establishing a replicable architecture for behavior-based analysis that future researchers and practitioners can extend.

Report Structure

The remainder of this report is organized as follows:

- **Section 3** presents theoretical background on behavior-based detection, containerization, and artificial intelligence in security analysis

- **Section 4** details the implementation of the Docker-based sandbox and dual-verdict pipeline
- **Section 5** describes the system interfaces and data flow
- **Section 6** presents testing results and validation of the prototype
- **Section 7** discusses findings, advantages, limitations, and implications
- **Section 8** concludes with recommendations for future work

2 Project Overview

2.1 Project Plan

2.1.1 Project Summary

Table 1: Summary information on the timeplan table (see Figure 1)

Field	Value
Project title	34249 Malware Detection
Project date shown	May 25, 2026
Project dates	Feb 2, 2026 – Jun 26, 2026
Completion	78%
Tasks	39
Resources	0

2.1.2 Top-Level Work Packages

Table 2: Top-level WBS items and major report milestone

Name	Begin date	End date
WBS 1: Local Sandbox	2/2/26	2/21/26
WBS 2: Docker Implementation	2/22/26	3/28/26
WBS 3: Initial Scalability Direction	3/29/26	4/25/26
WBS 4: Codebase Cleanup	4/27/26	4/27/26
WBS 5: Behaviour Analysis Prototype	4/28/26	6/18/26
WBS 6: Testing and Validation	4/30/26	5/15/26
Finish Midterm Report	4/28/26	5/29/26

Table 3: Scope pivot timeline

Milestone	Date
Working prototype with Kubernetes	4/19/26
Container runtime changed from Docker to containerd	4/23/26
Scope Pivot	4/26/26

2.1.3 Timeplan

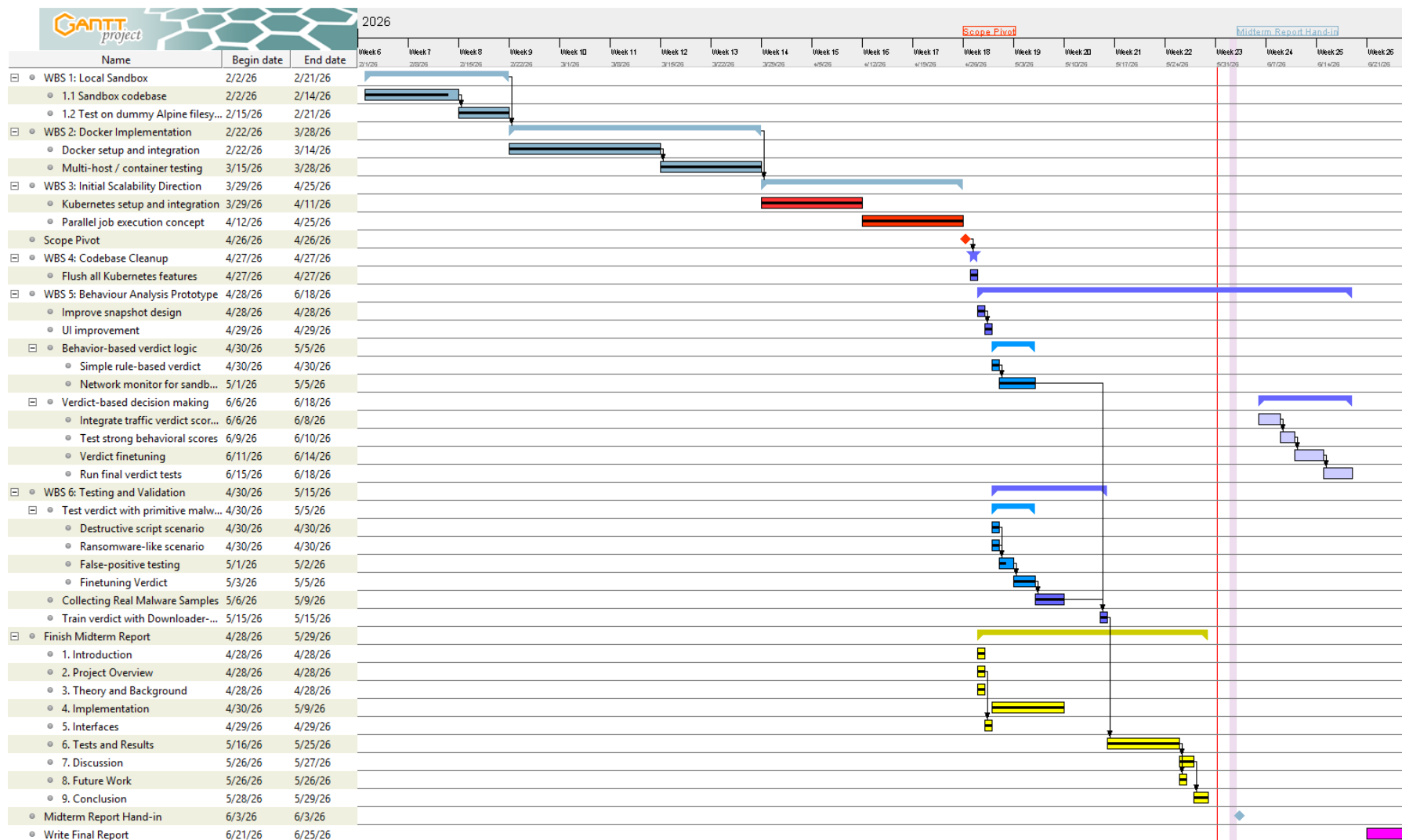


Figure 1: Project timeplan and development timeline

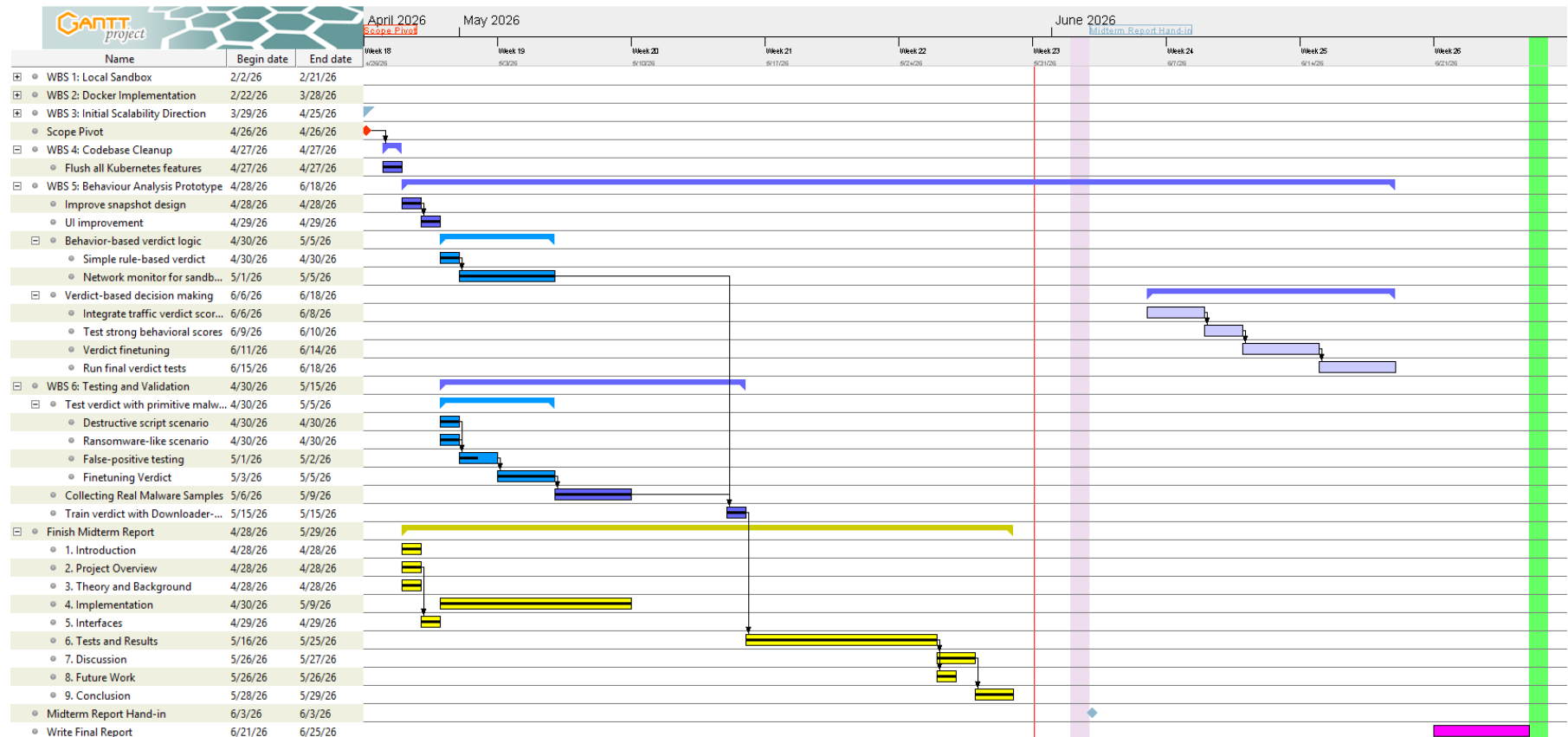


Figure 2: Timeplan zoom (26/04/2026 - 26/06/2026)

2.1.4 Tasks

Name	Begin date	End date
WBS 1: Local Sandbox	2/2/26	2/21/26
1.1 Sandbox codebase	2/2/26	2/14/26
1.2 Test on dummy Alpine filesystem	2/15/26	2/21/26
WBS 2: Docker Implementation	2/22/26	3/28/26
Docker setup and integration	2/22/26	3/14/26
Multi-host / container testing	3/15/26	3/28/26
WBS 3: Initial Scalability Direction	3/29/26	4/25/26
Kubernetes setup and integration	3/29/26	4/11/26
Parallel job execution concept	4/12/26	4/25/26
Scope Pivot	4/26/26	4/26/26
WBS 4: Codebase Cleanup	4/27/26	4/27/26
Flush all Kubernetes features	4/27/26	4/27/26
WBS 5: Behaviour Analysis Prototype	4/28/26	6/18/26
Improve snapshot design	4/28/26	4/28/26
UI improvement	4/29/26	4/29/26
Behavior-based verdict logic	4/30/26	5/5/26
Simple rule-based verdict	4/30/26	4/30/26
Network monitor for sandbox container	5/1/26	5/5/26
Verdict-based decision making	6/6/26	6/18/26
Integrate traffic verdict score with main verdict	6/6/26	6/8/26
Test strong behavioral scores	6/9/26	6/10/26
Verdict finetuning	6/11/26	6/14/26
Run final verdict tests	6/15/26	6/18/26
WBS 6: Testing and Validation	4/30/26	5/15/26
Test verdict with primitive malware	4/30/26	5/5/26
Destructive script scenario	4/30/26	4/30/26
Ransomware-like scenario	4/30/26	4/30/26
False-positive testing	5/1/26	5/2/26
Finetuning Verdict	5/3/26	5/5/26
Collecting Real Malware Samples	5/6/26	5/9/26
Train verdict with Downloader-Malware	5/15/26	5/15/26
Finish Midterm Report	4/28/26	5/29/26
1. Introduction	4/28/26	4/28/26
2. Project Overview	4/28/26	4/28/26
3. Theory and Background	4/28/26	4/28/26
4. Implementation	4/30/26	5/9/26
5. Interfaces	4/29/26	4/29/26

Name	Begin date	End date
6. Tests and Results	5/16/26	5/25/26
7. Discussion	5/26/26	5/27/26
8. Future Work	5/26/26	5/26/26
9. Conclusion	5/28/26	5/29/26
Midterm Report Hand-in	6/3/26	6/3/26
Write Final Report	6/21/26	6/25/26

Table 4: Task list extracted from the PDF

2.2 System Overview

The proposed system is designed to automatically analyze potentially malicious files in a controlled and isolated environment. It operates as part of a larger security workflow, where files from a target filesystem are identified, executed safely, and monitored for suspicious behavior.

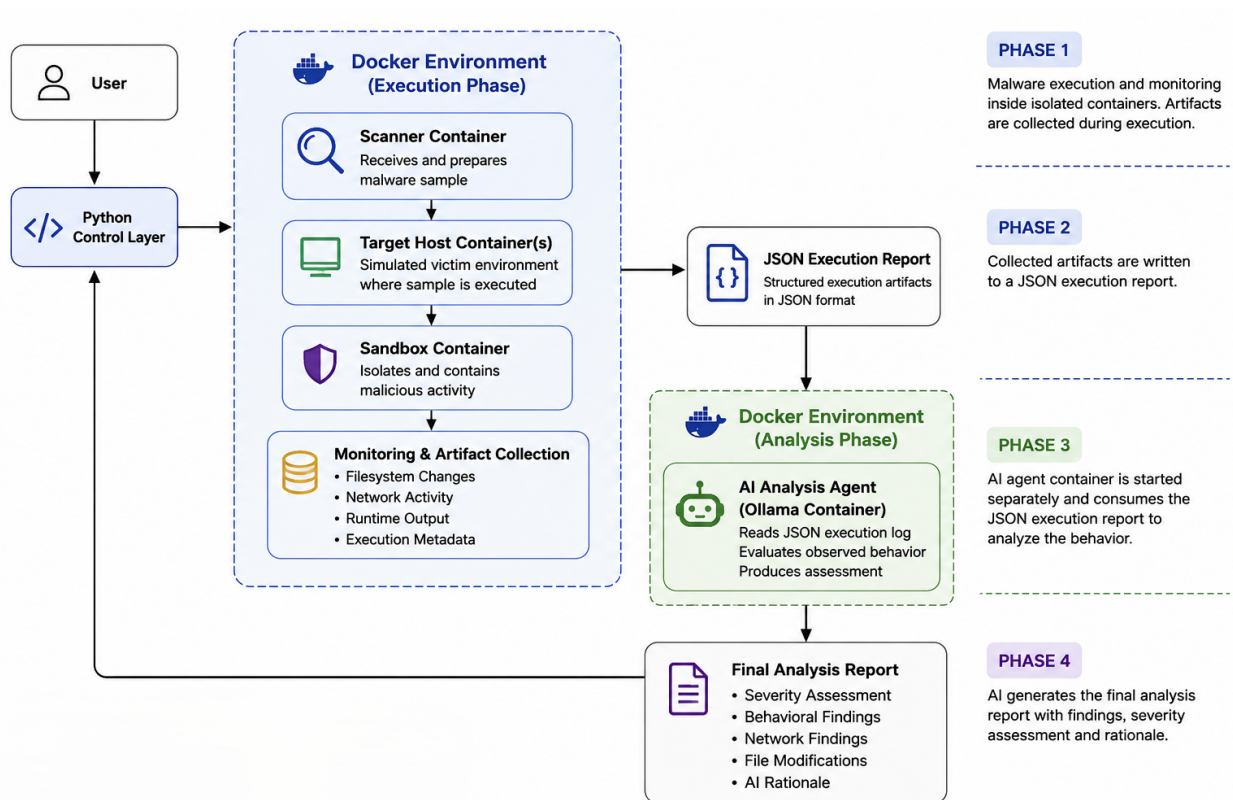


Figure 3: High-level system overview

The system integrates Python-based automation with Docker-based containerization to provide isolated execution and behavioral observation of suspicious files. The implemented prototype focuses on controlled malware analysis, artifact collection, and automated assessment of observed behavior.

2.2.1 Main Functionality

The primary purpose of the system is to detect malicious behavior by executing suspicious files in a sandboxed environment and analyzing their impact on the filesystem. This includes identifying file modifications, deletions, newly created files, and persistence mechanisms.

2.2.2 System Architecture and Subblocks

The system consists of several interacting components responsible for orchestration, sandbox execution, artifact collection, and behavioral analysis.

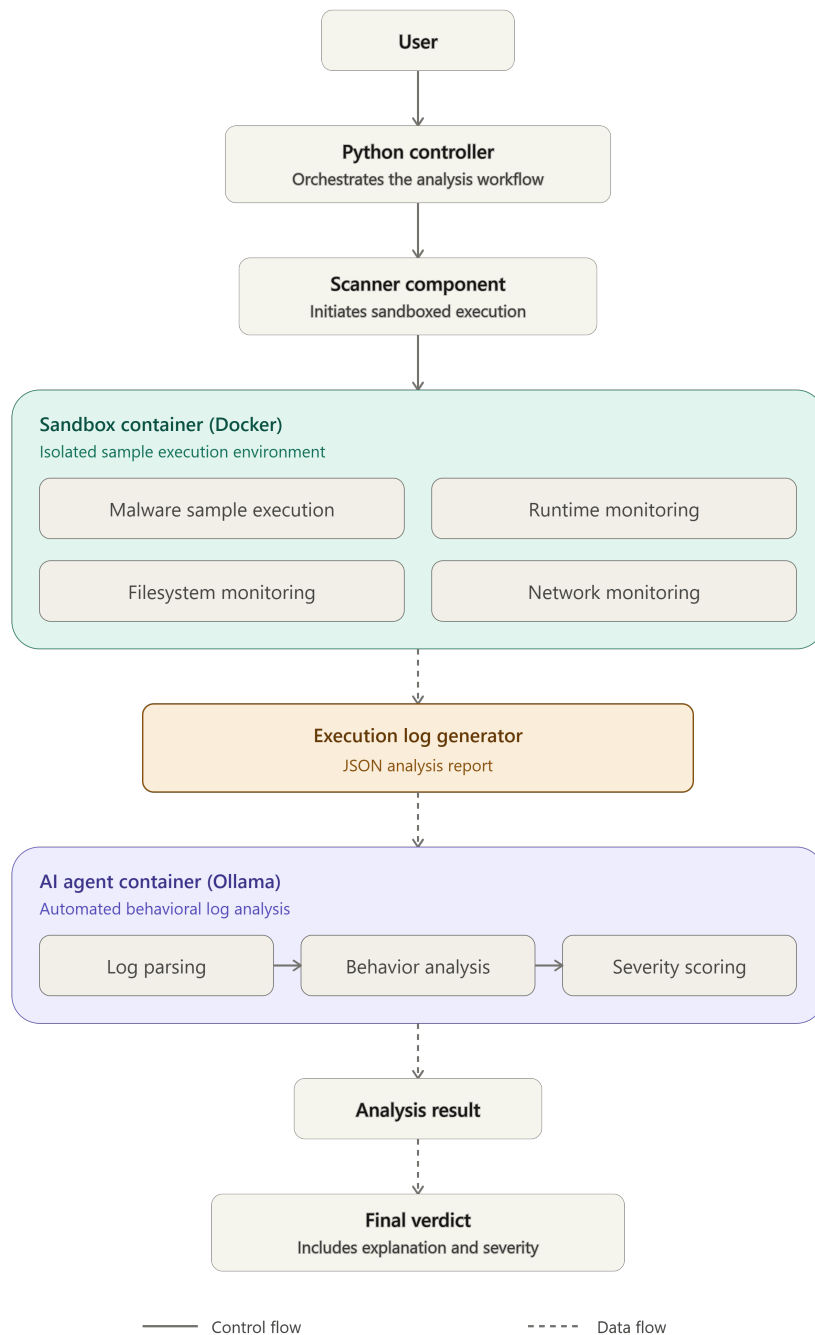


Figure 4: Conceptual architecture of the malware analysis platform

Figure 4 presents a high-level overview of the implemented system architecture. The figure illustrates how user interactions are handled by the Python control layer, which coordinates sandbox execution and monitoring through Docker-based containers. The collected execution artifacts are transformed into a structured JSON execution report and forwarded to an AI analysis agent, which produces the final behavioral assessment and severity classification.

The system is divided into four functional blocks:

- **Integration and Automation Block** – Consists of the User Interface, Python Control Layer, and orchestration logic responsible for coordinating the analysis workflow.
- **Docker-Based Execution Block** – Consists of the Scanner Container, Sandbox Container, and associated monitoring components responsible for isolated sample execution and artifact collection.
- **Analysis and Reporting Block** – Consists of the Execution Log Generator, JSON Execution Report, AI Agent Container, and Analysis Result components responsible for behavioral assessment and reporting.
- **Malware Sample and Test Block** – Represents the suspicious files, malware samples, and evaluation datasets processed by the platform during testing and validation.

Integration and Automation Block This block acts as the central control unit. It consists of Python-based automation scripts that orchestrate the overall workflow. The block is responsible for managing execution logic, coordinating container operations, distributing scan tasks, and collecting analysis results.

Docker-Based Execution Block The Docker-based execution block provides the isolated runtime environment used for malware analysis. It is responsible for:

- Building sandbox container images dynamically
- Deploying isolated containers for execution
- Running suspicious files inside controlled environments
- Collecting filesystem and runtime artifacts after execution

This block forms the core isolation mechanism of the implemented prototype.

Analysis and Reporting Block This block processes the artifacts collected during execution, including filesystem snapshots, network activity, runtime output, and execution metadata. The collected data is structured into a JSON execution report, which is analyzed by an AI agent to produce a behavioral assessment and final analysis report.

Malware Sample and Test Block This block is responsible for providing representative samples used during evaluation of the system. The samples include both manually authored malware-simulation scripts and collected real-world malware samples. This block works closely together with the *Behavior Analysis and Verdict Block*, supporting testing, calibration of the verdict logic, and validation of the behavior-based analysis workflow.

2.2.3 Scope and Requirements

The implemented prototype is designed to:

- Safely execute potentially malicious files in isolated Docker-based sandbox environments
- Monitor filesystem activity, network behavior, and runtime execution artifacts generated by executed samples
- Generate structured JSON execution reports describing observed behavior
- Support AI-assisted behavioral assessment of execution artifacts
- Provide interpretable analysis results for malware investigation and evaluation

The project focuses on behavior-based malware analysis in controlled environments rather than signature-based detection. The implemented prototype is intended for research and educational purposes and does not aim to replace production-grade malware-analysis platforms.

The design assumes access to Docker, a Linux-compatible operating system, and standard command-line utilities required for sandbox creation, filesystem inspection, and network monitoring.

3 Theory and Background

3.1 Behavior- vs Signature-Based Malware Detection

Malware detection methods are commonly divided into signature-based and behavior-based approaches. Signature-based detection is the most widely used technique in modern antivirus systems. It works by extracting unique patterns, such as byte sequences or file hashes, from known malware and comparing them against files in a system. This method is fast, efficient, and typically results in a low false positive rate. However, it relies heavily on previously identified malware and is therefore ineffective against new or unknown threats. Additionally, malware can evade detection by applying simple obfuscation techniques that alter its signature while preserving its functionality [4].

In contrast, behavior-based detection focuses on analyzing the actions performed by a program during execution. Suspicious files are executed in a controlled environment and monitored for malicious activities, such as unauthorized file modifications, system changes, or abnormal interactions with the operating system. If the observed behavior matches known malicious patterns, the file is classified as malware [4].

Unlike signature-based methods, behavior-based detection is capable of identifying previously unseen malware and variants that use obfuscation techniques. This makes it particularly useful for detecting zero-day attacks. However, this approach is more computationally expensive, as it requires runtime execution and monitoring, and may result in higher false positive rates due to the complexity of accurately distinguishing between benign and malicious behavior. Overall, while signature-based detection remains efficient for known threats, behavior-based detection provides a more robust solution for identifying emerging and evolving malware.

3.2 Static- vs Dynamic Analysis

Malware analysis techniques are commonly categorized into static and dynamic analysis. Static analysis examines a program without executing it, typically by analyzing its code, structure, or extracted features such as opcode sequences or API calls. This approach is generally fast and efficient, making it suitable for scanning large numbers of files. However, static analysis is limited in its ability to detect new or obfuscated malware, as it relies on observable code patterns that can be modified or hidden [5].

Dynamic analysis, on the other hand, involves executing a program in a controlled environment, such as a sandbox or virtual machine, to observe its behavior at runtime. During execution, various features can be monitored, including system calls, API calls, file system changes, and interactions with the operating system. This allows for a more comprehensive understanding of program behavior and enables the detection of previously unknown or obfuscated malware [5].

Studies have shown that dynamic analysis generally provides higher detection effectiveness compared to static methods, as it captures the actual execution behavior of a program. However, this comes at the cost of increased computational complexity and execution time, making it less scalable for large-scale analysis [5].

Overall, static analysis offers efficiency and speed, while dynamic analysis provides deeper insight into program behavior. As a result, dynamic techniques are often preferred when accurate detection of unknown threats is required, despite their higher resource demands.

3.3 Docker and Containers

Containerization is a lightweight virtualization approach that enables applications to run in isolated user-space environments called containers. A critical feature is process isolation, primarily achieved through Linux kernel primitives such as namespaces and control groups (cgroups). Namespaces partition system resources so that each container operates in its own isolated environment[6]. For example:

- **PID namespaces** isolate process IDs, ensuring processes inside a container cannot see or interact with processes in other containers.
- **Network namespaces** provide separate network interfaces, routing tables, and IP addresses.
- **Mount namespaces** isolate file system views, giving each container its own root filesystem.
- **User namespaces** map user and group IDs, enhancing security by reducing privilege exposure.

In addition to namespaces, cgroups enforce resource limits (CPU, memory, I/O), preventing containers from exhausting host resources. Together, these mechanisms ensure that containers are both isolated and efficient, combining strong separation with minimal overhead [7].

3.3.1 Docker

Docker is an open-source platform for developing, shipping, and running applications in containerized environments. It enables applications to be packaged together with their dependencies into lightweight containers, ensuring consistent execution across different systems. As a result, containers can run on any system with Docker installed, providing portability and efficient deployment[8].

3.4 Docker Networking

Docker provides several networking modes that determine how containers communicate with the host system and external networks. These networking modes are important in sandboxed malware analysis because they influence both isolation and the ability to observe network behavior.

3.4.1 Network Container Archetypes

Docker provides several networking configurations that determine how isolated a container is from the host system and other containers. According to Nikoloff [1], Docker networking can be viewed through four network container archetypes, illustrated in Figure 5. These archetypes range from highly isolated containers to containers with unrestricted access to the host network.

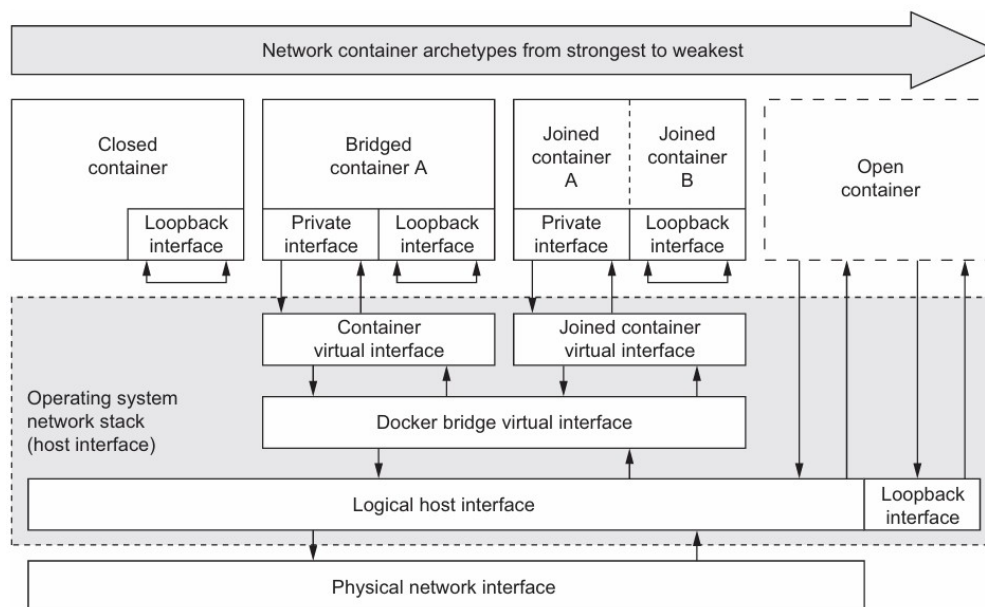


Figure 5: Four container network archetypes and their interaction with the Docker network topology [1]

The most restrictive archetype is the *closed container*, which only provides access to the internal loopback interface. Processes running inside the container can communicate with each other but cannot reach external systems or the internet. This provides the highest degree of network isolation.

The *bridged container* is Docker's default networking model. Containers are connected to a virtual bridge network that allows communication with external systems while maintaining separation from the host network. Containers on the same bridge network may also communicate with each other.

The *joined container* shares the network stack of another container. While file systems and process spaces remain isolated, both containers use the same network interfaces. This

configuration can be useful when multiple services must communicate through a shared network context.

The least restrictive archetype is the *open container*, which uses the host network directly. In this configuration the container has access to the host networking stack and therefore provides little network isolation. For security-sensitive workloads this configuration should generally be avoided.

As shown in Figure 5, the archetypes can be interpreted as a spectrum ranging from maximum isolation to maximum connectivity. The choice of networking model depends on the desired balance between security, observability, and functionality.

3.4.2 Docker Network Drivers

Docker implements networking through a set of network drivers that determine how containers are connected to each other and to external networks.

- **Bridge** – The default Docker network driver. Containers are connected through a virtual bridge (`docker0`) and can communicate through a private subnet.
- **Host** – The container shares the host's network stack directly, corresponding to the open container archetype.
- **None** – No external network interfaces are created, corresponding to the closed container archetype.
- **Overlay** – Enables communication between containers across multiple hosts and is commonly used in orchestration platforms.
- **Macvlan** – Assigns containers their own MAC addresses, allowing them to appear as independent devices on the network.

The malware-analysis platform developed in this project primarily relies on bridged networking. This configuration provides a balance between containment and observability by allowing malware samples to communicate through a controlled network interface while enabling network activity to be monitored and analyzed.

3.5 Container-Based Malware Analysis

Containers provide a lightweight and reproducible environment for executing potentially malicious software while limiting its ability to affect the host system. Unlike traditional virtual machines, containers do not emulate an entire operating system. Instead, they rely on Linux isolation mechanisms such as namespaces, control groups (cgroups), capability restrictions, and user separation to isolate processes from the surrounding environment [2].

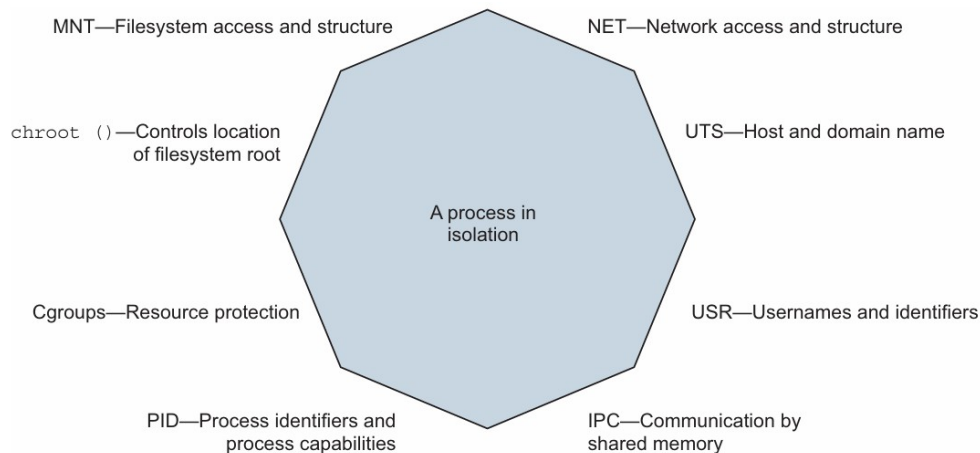


Figure 6: Linux isolation mechanisms used to construct Docker containers [2]

Figure 6 illustrates the primary isolation mechanisms used by Docker containers. Rather than relying on hardware virtualization, Docker combines several Linux kernel features to create isolated execution environments. These mechanisms control access to filesystems, networking resources, process identifiers, shared memory, user identities, and system resources, while simultaneously limiting the privileges available to processes executing inside the container.

Among the isolation mechanisms shown in Figure 6, several are particularly relevant to malware analysis. The *MNT* namespace isolates filesystem views between the host and the container, enabling potentially malicious software to modify files within the sandbox without directly affecting the host system. The *NET* namespace provides independent network interfaces and routing tables, allowing network activity to be monitored and controlled. The *PID* namespace isolates process identifiers and process visibility, preventing malware from interacting with processes outside the container. Finally, cgroups enforce resource limits such as CPU and memory consumption, helping prevent malicious software from exhausting host resources [2].

These security mechanisms make containers attractive for malware analysis. Suspicious files can be executed within a controlled environment where filesystem modifications, persistence attempts, process creation, and network activity can be observed without granting direct access to the host system. Furthermore, containers can be created, destroyed, and recreated rapidly, enabling automated and repeatable analysis workflows.

However, container-based malware analysis also has limitations. Because containers share the host kernel, they generally provide weaker isolation than full virtual machines. Vulnerabilities in the container runtime, kernel, or container configuration may allow container escape attacks, where malicious code gains access to resources outside the intended security boundary. As a result, containerized malware analysis systems should follow the principle of least privilege and avoid granting unnecessary capabilities or privileged execution modes [2].

Another challenge is the existence of environment-aware malware. Modern malware increasingly incorporates anti-analysis techniques designed to detect whether it is executing inside a sandbox, virtual machine, or containerized environment. Upon detecting such conditions, malware may alter its behaviour, delay execution, disable malicious functionality, or attempt to evade detection entirely. Research has also demonstrated that container environments introduce their own security challenges, including vulnerabilities in container images and runtime configurations that may be exploited by attackers [9].

Despite these limitations, containerized execution provides an effective balance between isolation, performance, and operational simplicity. For behavior-based malware analysis, where large numbers of suspicious files may need to be executed and monitored automatically, containers offer significantly lower overhead than traditional virtual machines while still providing sufficient containment and observability for many analysis scenarios.

3.6 Artificial Intelligence in Cybersecurity and Malware Detection

Traditional malware detection relies on signature-based approaches, which match known malicious patterns against incoming files. While effective against known threats, signature-based detection cannot address zero-day exploits or polymorphic malware that intentionally varies its structure to evade detection [10]. Behavioral analysis, in contrast, observes what a program *does* rather than what it *is*, making it robust against structural variations while remaining semantically meaningful.

The shift toward machine learning-based malware detection has been extensively documented. Saxe and Berlin demonstrated that deep learning approaches can achieve high accuracy on malware classification tasks, with the key insight being that malicious behavior exhibits consistent patterns across structural variations [11]. Their work established that neural networks can learn behavioral features that generalize well to previously unseen malware families—a critical advantage for detecting novel threats.

Recent research has identified specific behavioral indicators that reveal malicious intent:

- **File System Behavior:** Unauthorized modifications to sensitive system directories, file deletion patterns consistent with covering tracks, and creation of executable payloads in non-standard locations [12]
- **Network Behavior:** Command-and-control (C2) communications, data exfiltration to suspicious IP addresses, and connections to known malicious domains [13]

- **Process Behavior:** Privilege escalation attempts, process injection, and spawning of suspicious child processes [14]
- **Persistence Mechanisms:** Registry modifications, scheduled task creation, and service installation for maintaining access across reboots [15]

Critically, these behaviors are *contextual*—their meaning depends on combinations and sequences. A single network connection might be benign; however, a connection immediately following file encryption and deletion strongly suggests ransomware behavior. Traditional rule-based systems struggle with such contextual combinations because they require explicit threshold specification for each combination, leading to brittle systems that fail against novel attack patterns.

AI-driven analysis addresses this limitation through learned contextual understanding. Unlike rules, which require manual engineering of thresholds for each behavioral combination, machine learning models implicitly learn which behavioral patterns co-occur in malicious samples and which are benign. This learned understanding generalizes better to novel malware variants [16].

3.7 AI Agents for Behavioral Assessment

An AI agent is a system that perceives its environment, reasons about observations, and takes actions based on that reasoning [17]. In the context of malware analysis, the agent operates in a well-defined loop:

1. **Observe:** Receive execution artifacts (file system changes, network logs, process behavior)
2. **Reason:** Apply learned patterns and contextual understanding to these artifacts
3. **Decide:** Assess whether observed behavior is consistent with malicious intent
4. **Act:** Generate a verdict with confidence assessment

Recent advances in large language models (LLMs) have demonstrated that these systems excel at reasoning over sequential, contextual information—precisely the format of execution logs from dynamic malware analysis [18]. LLMs achieve this through transformer-based architectures that use attention mechanisms to model relationships between behavioral elements, allowing them to implicitly weight which behaviors are most significant in determining maliciousness [19].

The specific advantage of LLM-based agents for security analysis is **in-context learning**: these models can understand new behavioral patterns from their training data without requiring retraining on new malware samples. When presented with a complete execution log in structured format (JSON), an LLM agent can reason about whether the combined behaviors indicate malicious intent, drawing on knowledge of malware tactics, techniques, and procedures (TTPs) learned during pre-training.

Ollama provides a critical capability for security applications: local LLM inference without external API calls. This addresses the privacy concern inherent in cloud-based security analysis—sensitive execution logs containing filesystem, network, and process information never leave the analysis environment. Locally-run models provide deterministic behavior suitable for forensic analysis while maintaining full confidentiality of analyzed samples.

3.8 Trust, Explainability, and the Black Box Problem

Machine learning systems, particularly deep neural networks and LLMs, are often criticized as “black boxes”—systems that produce decisions without transparent reasoning. In security applications, this opacity is problematic: when a system flags a file as malicious, security analysts need to understand *why* to verify correctness and refine detection logic [20].

The explainability challenge in AI-based malware detection is well-documented. Demetrio et al. demonstrated that adversarial examples—inputs specifically crafted to fool machine learning models—can be generated against malware detectors, and these evasion techniques remain applicable even when the underlying model is unknown [21]. This adversarial robustness problem means that opaque AI systems introduce a new attack surface: malware authors can attempt to craft samples that bypass AI detection without understanding the model’s internals.

Several approaches mitigate the trust and explainability problem:

Hybrid Validation

Compare AI verdicts against rule-based baselines. Disagreement between AI and rule-based systems triggers deeper analysis rather than accepting either verdict uncritically. This approach combines the transparency of rules (analysts understand each decision criterion) with the generalization of AI (learning from data rather than manual specification) [22].

Testing and Validation

Systematically evaluate AI-based verdicts on:

- Known malware samples with confirmed threat classifications
- Benign samples from diverse software ecosystems
- Borderline cases where behavior is ambiguous

This provides empirical evidence of detection accuracy and false positive/negative rates—the foundation for trustworthiness in security systems [23].

Confidence Scoring

Rather than binary “malicious/benign” decisions, AI agents can provide confidence scores reflecting uncertainty. A score of 0.95 (highly confident) carries different weight than 0.55 (uncertain), allowing analysts to adjust investigation depth based on model confidence [24].

Explanation Generation

Recent LLM architectures support “chain-of-thought” reasoning where the AI explicitly states intermediate reasoning steps. For example, rather than simply reporting “malicious,” the model can report: “File created in protected system directory + multiple network connections to non-standard ports + process injection detected = consistent with agent installation.” This reasoning is still AI-generated but more transparent than unexplained verdicts [25].

Adversarial Robustness

For critical security applications, consider ensemble methods combining multiple detection approaches (AI + rules + heuristics) and periodic retraining on new malware samples to maintain effectiveness as malware evolves [26].

The goal is not to eliminate the use of AI in security, but rather to deploy it responsibly by acknowledging its limitations, validating its decisions empirically, and maintaining human oversight of critical verdicts.

4 Implementation

This section describes the implementation of the project prototype as it exists in its current stage. The final solution focuses on Docker-based isolation and behavior-based malware detection. Earlier design ideas focused on the infrastructure, where Kubernetes was a key component, and used for orchestration. However, this was not pursued further, as it introduced unnecessary complexity for the current project scope. The implemented system instead emphasizes controlled execution, observation of malicious behavior inside containers, and decision-making based on behavior analysis. The purpose of this section is to explain the architecture and internal logic of the system; concrete evaluation scenarios and test-specific setup are described separately in the testing part of the report.

4.1 Implementation Strategy

The implementation is divided into two main parts: a Python control layer and a Docker-based execution environment. Python is used to coordinate the workflow, while Docker provides the isolation required to execute suspicious files without running them directly on the host system.

This design supports two practical goals. First, it allows suspicious files to be executed in a realistic Linux environment rather than directly on the host system. Second, it enables the system to measure behavioral effects, such as file creation, deletion, modification, and changes to sensitive system paths.

4.2 Docker-Based Environment

The project uses four separate Docker images, each responsible for a specific part of the analysis workflow:

- A **host image**, which represents the machine being analyzed
- A **scanner image**, which contains the Python scanning logic and communicates with Docker through the mounted Docker socket
- A **sandbox image**, which is the sandbox in which the files are executed
- An **ollama image**, which provides the large language model inference capability used by the AI agent for verdict generation

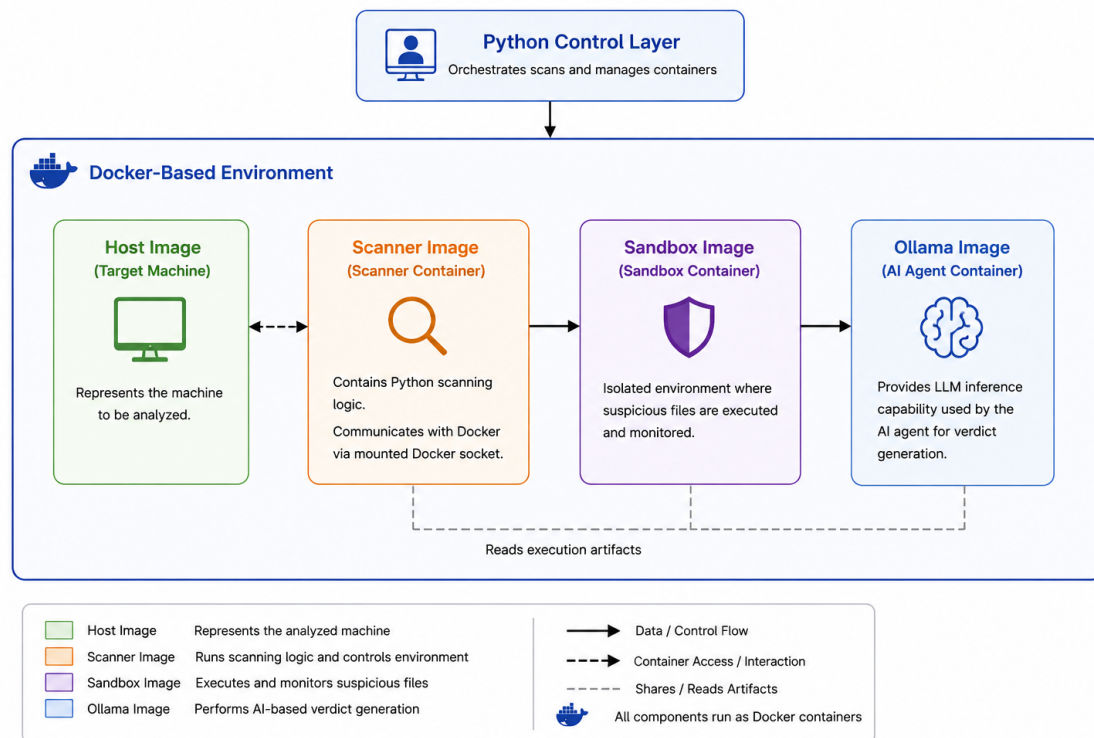


Figure 7: Overview of the Docker-based environment and the interaction between the host, scanner, sandbox, and Ollama containers.

As illustrated in Figure 7, the scanner container acts as the central orchestration component. It interacts with the host container to discover candidate files, generates sandbox environments for isolated execution, and collects the resulting execution artifacts. Once execution has completed, the collected artifacts are passed to the Ollama container, which performs large language model inference and assists the AI agent in generating the final assessment.

The host image is defined in the project under `ContainerImages/Host/Dockerfile`. It creates a lightweight Alpine-based environment with a simulated user filesystem and predefined directory structure. This image acts as the target system that the scanner interacts with during analysis.

The scanner image is defined in `ContainerImages/Scanner/Dockerfile`. It contains the Python scripts responsible for locating executable files, launching isolated analysis containers, collecting runtime artifacts, and coordinating the subsequent AI-based analysis process.

The sandbox image is not defined through a static Dockerfile. Instead, it is generated dynamically by `linux_container.py`. In contrast to the host and scanner containers, which remain relatively static throughout execution, the sandbox image is rebuilt for each analysis iteration. This allows the sandbox environment to mirror the selected target path and include only the files required for the current execution scenario.

The **Ollama image** provides the large language model inference backend used by the AI agent. After the scanner has completed execution and generated the structured JSON execution report, the AI agent invokes the Ollama container to analyze the collected behavioral artifacts and produce the final assessment.

4.3 Python Control Layer

The implementation is orchestrated through a set of Python scripts in the project root. The file `main.py` provides a command-line interface for the overall workflow. From this interface, the user can create analysis hosts, list running hosts, run the scanner against a local path, run the scanner against a container path, distribute scans across directories, and copy selected malware samples into a target host.

4.3.1 Execution Pipeline

The supporting logic is implemented in `helpers.py`. This module handles container discovery, host creation, malware sample listing, file copying into containers, and construction of the Docker commands used to run the scanner image. In practice, this module acts as the integration layer between Python and the Docker runtime, forming an efficient pipeline, making it easy to use and allows for rapid-testing.

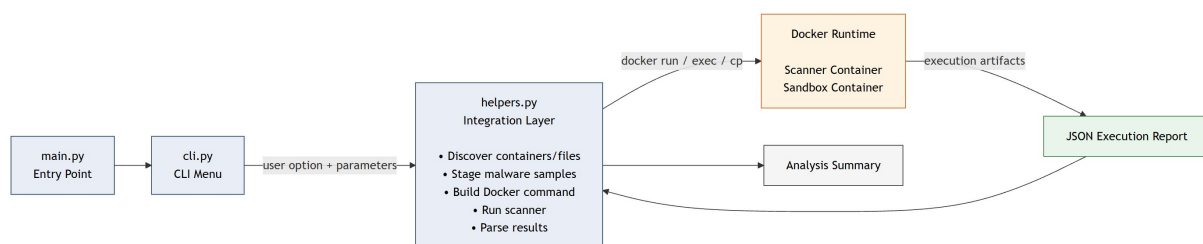


Figure 8: Simplified workflow showing module interaction and data flow through the execution pipeline. `helpers.py` acts as the central integration point between Python application logic and the Docker runtime.

The execution flow begins when `main.py` initializes the system and passes control to `cli.py`, which presents a menu of operations to the user (as shown in Figure 8). Based on the selected option and provided parameters, `helpers.py` orchestrates the workflow: discovering available containers and files, optionally staging malware samples into the target environment, constructing the appropriate Docker command, and executing the scanner via the Docker runtime. The Docker containers—both the scanner image and the sandbox container—produce execution artifacts including JSON reports and filesystem snapshots. These results are parsed and summarized before being presented to the user, completing the cycle.

4.3.2 Kubernetes Substitute

The file `distributed_scan.py` adds a simple parallel scanning mechanism. Instead of relying on Kubernetes, the project distributes work by enumerating directories inside a selected container and launching multiple scanner jobs from Python using a thread pool. This provides a lightweight form of workload distribution within the scope of the prototype without introducing a full orchestration platform.

4.4 Scanning and Execution Workflow

The dynamic analysis logic is implemented in `linux_sandbox.py` and `linux_container.py`. The first script is responsible for scanning a target path and identifying executable files based on a set of relevant extensions such as `.sh`, `.py`, `.bin`, and `.run`. When such a file is found, it is passed to the container execution stage.

The execution stage creates a temporary sandbox based on Ubuntu. The relevant files are copied into the sandbox, the selected sample is made executable, and the file is run inside an isolated Docker container. Before execution, the system creates a manifest of the files under `/home` using SHA-256 hashes. After execution, the same manifest is collected again. By comparing the two manifests, the system can detect modified, newly created, and deleted files.

In addition to the manifest comparison, the implementation also uses `docker diff` to inspect filesystem changes at the container level. This is important because some malicious actions may target locations outside the monitored user directory, for example by creating files in `/opt` or altering service-related files under `/etc`. Combining the two methods improves visibility into both user-space and system-level effects.

4.5 Verdict Logic

The verdict system has evolved from an initial rule-based approach to a modern AI agent-based approach. Both methods are available in the implementation and provide complementary perspectives on malware classification.

4.5.1 Rule-Based Verdict (Legacy Approach)

The original decision logic is implemented in `verdict.py`. Rather than classifying a sample from a static signature, this approach assigns a verdict based on the behavior observed during execution. The scoring logic considers several indicators:

- File deletions, especially when multiple files are removed
- Modifications to document-oriented user files
- Creation or modification of executable content
- Access to sensitive system paths such as `/etc`, `/bin`, or `/usr`

- Suspicious network-related activity detected through `tcpdump_mon.py` and analyzed by `traffic_verdict.py` using AbuseIPDB reputation scoring
- Indicators of encryption-like behavior

The network monitoring component allows the system to observe outbound and inbound connections made during execution. Observed IP addresses are analyzed using the AbuseIPDB API in order to identify potentially malicious or highly reported hosts. This extends the verdict system beyond filesystem-based behavior and adds an additional layer of behavioral analysis.

Based on the accumulated score, a sample is classified as *benign*, *low-confidence suspicious*, *suspicious*, or *likely malicious*. This deterministic approach explicitly operationalizes behavioral indicators and provides transparency in the decision-making process.

4.5.2 AI Agent-Based Verdict (Current Approach)

The current implementation supplements the rule-based system with an AI agent-based verdict mechanism. This approach leverages machine learning and natural language reasoning to analyze the collected behavioral artifacts more holistically. Rather than relying solely on hand-crafted thresholds, the AI agent ingests the full context of runtime observations—including filesystem changes, network traffic, process behavior, and timing patterns—and synthesizes a verdict based on learned patterns of malicious behavior.

The AI agent approach provides several advantages over the legacy rule-based method:

- **Contextual reasoning:** The agent can identify complex behavioral signatures that emerge from combinations of simpler indicators, capturing attack patterns that explicit rules might miss.
- **Adaptability:** The agent can generalize to novel malware variants without requiring manual rule updates.
- **Reduced false positives:** By considering the broader context of observed behavior, the agent can distinguish between benign programs with unusual behavior and actual malicious activity.
- **Explainability:** The agent can provide rationale for its classification decisions, enhancing interpretability compared to opaque rule combinations.

Both approaches are evaluated in the testing section to demonstrate their relative strengths and to quantify the improvement provided by the AI agent-based method.

4.6 Malware Behavior Focus

The implemented prototype is centered around representative malware behaviors rather than complex deployment infrastructure. The main emphasis is on detecting practical indicators of compromise through runtime observation, not on building a large orchestration layer.

Examples of these behaviors include destructive actions, document modification, creation of executable payloads, and persistence-related changes in system directories. This focus on observable behavior makes the prototype more relevant to malware analysis than a broader but less developed orchestration setup would have been.

4.7 Ollama Wrapper Integration

To support AI-assisted behavioral assessment, the implementation includes a lightweight Python-based Ollama wrapper that acts as the interface between the malware-analysis pipeline and the local language model runtime. Its purpose is to transform the collected execution artifacts into a structured prompt, submit this prompt to the selected local model through Ollama, and return the generated assessment in a predictable format for further processing. This wrapper separates model interaction from the rest of the control logic, making it easier to test different prompts, models, and output formats without changing the surrounding sandbox and monitoring components. By keeping inference local, the design also preserves confidentiality, since execution logs and behavioral data remain inside the analysis environment rather than being transmitted to external services.

4.8 Summary

The current implementation reflects a deliberate shift from orchestration-oriented design toward a smaller and more focused malware-analysis prototype. Docker is used as the primary isolation mechanism, Python provides coordination and automation, and the detection logic is based on behavioral evidence collected during runtime. The system now supports both rule-based and AI agent-based verdicts, allowing direct comparison of traditional and modern approaches to malware classification. As a result, the project better matches its main objective: analyzing suspicious files by observing what they do rather than relying on how they look.

5 Interfaces

This section describes how the implemented parts of the prototype interface with each other and how data moves between them during a scan. In the current version of the project, the architecture is centered around two main blocks: a Python control layer and a Docker-based execution layer. Kubernetes was part of an earlier design direction, but it is not part of the implemented prototype and is therefore not treated as an active interface in this report.

The purpose of the interface description is not only to list technologies, but also to explain how the blocks cooperate in practice. In this project, the interfaces are mainly software interfaces based on command-line invocation, Docker runtime operations, file transfer, and generated scan artifacts.

5.1 Overview of the Implemented Interfaces

At a high level, the interface structure is simple. The Python control layer receives user input, translates that input into scanner actions, and invokes Docker commands. The Docker-based execution layer then creates or reuses containers, copies relevant files into an isolated sandbox, runs the selected sample, and returns execution artifacts to the Python side for interpretation.

This means that the most important interfaces in the current implementation are:

- The user-facing command-line interface
- The interface between Python and the Docker runtime
- The interface between the scanner container and the generated sandbox container
- The filesystem interface used to copy input data and collect output artifacts
- The result interface through which verdicts and observations are returned

5.1.1 Interface Workflow

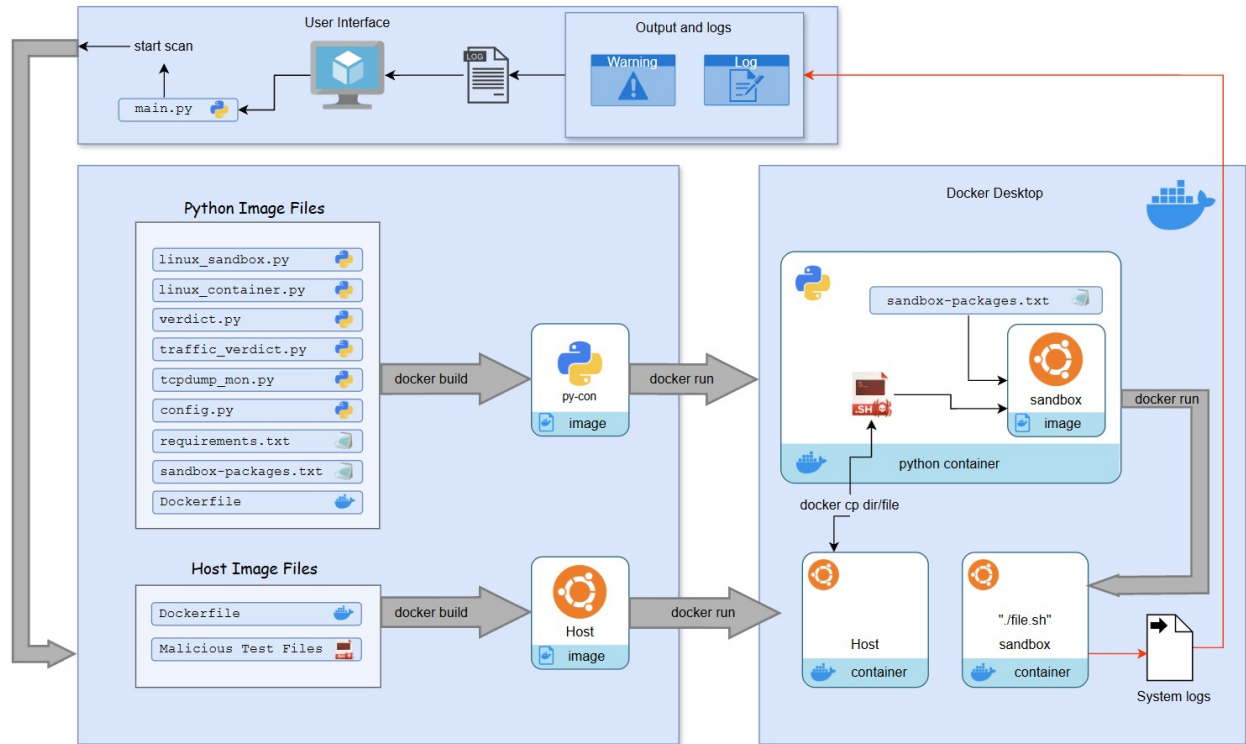


Figure 9: Interface Workflow

Figure 9 illustrates the workflow and interface relationships involved in sample preparation, sandbox execution, runtime monitoring, and artifact collection. The workflow begins with user interaction through the Python control layer, which coordinates Docker operations and scanner execution. The scanner prepares and launches isolated sandbox containers, executes the selected sample, collects execution artifacts, and returns the resulting data for further analysis.

The figure focuses on the execution pipeline implemented by the scanner and sandbox environments and serves as a roadmap for the interfaces described in Sections 5.2–5.6. The workflow concludes with the generation of execution artifacts, which are subsequently processed by the analysis interfaces described in the following sections.

5.2 User Interface to the Python Control Layer

The first interface is the user-facing command-line interface provided by the Python control layer. This interface is implemented through `main.py`, which presents a menu where the user can choose actions such as creating host containers, selecting a path for scanning, starting a container-based scan, distributing scan work, or uploading malware samples into a host container.

Functionally, this interface converts user choices into structured execution parameters. These parameters include, for example, a target path, a target container identifier, or a specific upload source. The Python layer then maps these choices to internal function calls and finally to Docker commands. In this sense, the command-line interface acts as the entry point to the complete scanning workflow. As illustrated in Figure 9, this interface represents the entry point to the malware-analysis workflow and is responsible for initiating all subsequent processing stages.

The data format at this interface is simple text input and terminal output. This makes the interface easy to test and easy to modify during development, even though it is not graphical.

5.3 Python Control Layer to Docker Runtime

The most central internal interface in the prototype is the interface between Python and Docker. This interface is implemented through Python subprocess calls that invoke Docker commands such as `docker run`, `docker exec`, `docker cp`, `docker diff`, `docker build`, and `docker ps`.

This interface is responsible for translating high-level scanning logic into concrete container operations. Within the workflow shown in Figure 9, this interface forms the bridge between orchestration logic and the containerized execution environment. For example, when the user requests a scan of a path inside a host container, the Python layer:

- Discovers the relevant host container
- Constructs the scanner command
- Launches the scanner container with access to the Docker socket
- Passes the target path and optional container identifier as command-line arguments

From an interface perspective, the Python block does not communicate with Docker through a custom network protocol. Instead, it relies on the Docker command-line interface as the control boundary. The mounted Docker socket makes it possible for the scanner container to continue this interaction and create additional containers dynamically.

This interface is important because it defines the practical separation between orchestration logic and execution environment. Python decides *what* should happen, while Docker performs *where* and *how* the suspicious file is isolated and executed.

5.4 Scanner Interface to the Sandbox Environment

Inside the Docker-based part of the system, there is a second important internal interface: the interface between the scanner logic and the generated sandbox container. This interface

is implemented in two stages. As shown in Figure 9, the scanner acts as an intermediary component between orchestration and execution, preparing the sandbox environment and controlling sample execution.

First, the scanner identifies a candidate executable file and prepares a temporary filesystem snapshot. Depending on the scan mode, this is either a local directory copy or a curated snapshot of relevant paths copied from a target host container. Second, this prepared filesystem is embedded into a newly generated sandbox image, and the suspicious file is executed inside that container.

The interface between these parts consists of:

- The copied filesystem content that becomes the sandbox input
- The generated Dockerfile that defines the sandbox image
- The selected sample path passed to the sandbox execution logic
- Execution control through `docker create`, `docker start`, and `docker exec`

This interface is effectively the boundary between preparation and analysis. The scanner prepares the environment and the sandbox executes the file under controlled conditions. Because the sandbox is rebuilt for each sample, the interface is highly dynamic, but still deterministic in structure.

5.5 Filesystem and Artifact Interface

The filesystem interface is central to the behavior-based detection method. This stage corresponds to the artifact-collection phase illustrated in Figure 9, where execution results are transformed into structured analysis inputs. It is through this interface that the system both injects input data into the sandbox and retrieves analysis artifacts after execution.

Before execution, the relevant files are copied into the sandbox context. After execution, the system retrieves several artifacts:

- The malware standard output and error output
- A manifest of files before execution
- A manifest of files after execution
- Container-level filesystem differences from `docker diff`
- Network-monitor output generated during execution
- The recorded sample exit code

The manifest files form a simple but effective internal data format. Each line stores a path and its hash value, which makes it possible to compare the state before and after execution and identify modified, new, and deleted files. This is one of the clearest examples of an intracommunication interface in the implemented prototype: one part of the system produces structured artifacts, and another part consumes them to generate a verdict.

5.6 Runtime Monitoring Interface

The project also includes an interface for runtime observation. The monitoring path shown in Figure 9 operates alongside sample execution and provides supplementary behavioral data that can be incorporated into later analysis. During sample execution, the scanner starts a separate monitoring process that captures traffic from inside the sandbox container. This monitoring path is implemented through `tcpdump_mon.py`, which is started by the scanner and attached to the running sandbox container.

The monitor communicates its result back through a generated summary artifact rather than through direct interactive messaging. More specifically, the scanner starts the monitor before execution, stops it afterward by creating a stop-file, and then reads the final connection summary produced by the monitor. This creates a simple synchronization interface between execution and observation.

The exchanged monitoring data includes remote IP addresses, ports, protocol information, packet counts, and payload counters. These values are then rendered in the scanner output and can also contribute to traffic-related interpretation of the sample behavior.

5.7 Analysis Output Interface

Once the execution artifacts have been collected, they are forwarded to the analysis subsystem. This interface forms the boundary between raw execution observations and the final behavioral assessment generated from those observations.

The verdict component receives:

- Modified, new, and deleted file lists
- Filtered container diff paths
- Malware output text
- Optional network-related observations

Based on these inputs, the verdict logic assigns a score and a label such as *benign*, *low-confidence suspicious*, *suspicious*, or *likely malicious*. The output is then returned to the terminal in a human-readable format. In distributed scan modes, machine-readable result markers are also emitted so that the Python layer can aggregate results across multiple runs. This makes the output interface both user-oriented and script-oriented.

5.8 Execution Log Interface

The scanner generates a structured JSON execution report capturing the complete sandbox session. Filesystem mutations are detected via Docker's `docker diff` command, which compares the container state before and after execution, partitioning changes into modified, new, and deleted files. The report also includes target file metadata, execution time, exit code, network connections, and captured runtime output from the sample.

This JSON artifact serves as the primary interface between the sandbox environment and the analysis subsystem. By exposing execution results in a structured and machine-readable format, the report can be consumed by automated analysis components, threat intelligence workflows, or AI-based classification systems. In the implemented prototype, the generated execution log is forwarded to a dedicated AI agent responsible for assessing the observed behavior and generating a severity classification.

5.9 AI Analysis Interface

The AI analysis interface provides an alternative classification path that complements the rule-based verdict system. The execution log JSON generated by the sandbox environment is forwarded to an AI agent running inside a dedicated ollama container. This agent ingests the complete behavioral record—including filesystem mutations, network connections, process output, and execution metadata—and applies learned patterns to synthesize a holistic verdict on the sample's maliciousness. Unlike the deterministic rule-based approach, which applies fixed thresholds to individual indicators, the AI agent reasons about the contextual relationships between behaviors, enabling it to detect subtle or novel malware signatures that a combination of simple rules might miss. The agent's assessment is returned as structured output that includes both a classification label and supporting rationale, making the AI-driven verdict both actionable and interpretable.

5.10 Assumptions and Interface Constraints

The implemented interfaces rely on several practical assumptions. The system assumes a Linux-compatible environment, access to Docker, permission to mount the Docker socket into the scanner container, and support for standard command-line tools such as `find`, `sha256sum`, and `tcpdump`.

The design also assumes that communication between blocks is local and trusted, since the prototype is not built as a remote service with authentication or encrypted transport. For the current project scope, this is acceptable because the goal is controlled malware analysis on a lab-style system rather than deployment as a production security platform.

Another important limitation is that the project does not define a formal communication protocol between blocks in the network-engineering sense. Instead, the interfaces are implemented through operating-system commands, Docker runtime actions, file-based artifacts, and structured terminal output. For this prototype, those interfaces are sufficient and directly reflect the implemented system.

5.11 Summary

The current prototype has a small number of interfaces, but they are central to the system design. The user interacts with the Python control layer, the Python layer controls Docker, the scanner prepares and launches isolated sandbox containers, and the resulting execution artifacts flow back into the verdict logic. In that sense, the interface structure mirrors the complete malware-analysis pipeline: input selection, isolated execution, observation, and classification.

6 Tests and Early Results

This section presents the first test scenarios used to evaluate the current prototype. The goal at this stage is not to provide a complete validation of the final system, but to demonstrate that the implemented Docker-based analysis workflow can execute suspicious files in isolation and detect relevant behavioral changes.

6.1 Test Setup

The tests are performed using the implemented prototype described in the previous sections. A host container is used as the target environment, while the scanner container is responsible for locating suspicious files, launching sandboxed execution, and collecting behavioral evidence. The analysis focuses on observable filesystem changes before and after execution, supplemented by container-level inspection.

6.2 Test Scenarios

The current evaluation is centered around representative malware-like behaviors that are relevant to a behavior-based detection system. The scenarios include:

- Early stage testing included:
 - Destructive "evil" scripts that delete files
 - Ransomware-like behavior that modifies (encrypts) user files
 - C2-Downloader, which downloads and deploys an agent (creates a process), hereby touching sensitive paths such as `systemd`
- Testing the verdict
 - Test primitive/obvious malware on the verdict
 - Test downloader scripts on the network traffic verdict
- Collect and test malware samples
- Test a real, unfiltered malware program

These scenarios were chosen because they produce concrete system effects, and is definitive for the different iterations of the project prototype.

6.3 Early-stage Testing

6.3.1 Destructive Script Simulation

```
[+] Executable detected: ../alpine-rootfs/home/user/Desktop/evil.sh
[+] Launching sandbox for: /mnt/c/Users/Lauri/Desktop/Python/Anitvirus/alpine-rootfs/home/user/Desktop/evil.sh
[+] Building sandbox image...
[+] Starting isolated container...
[+] Container ID: 9a58cea2e3f85c794e84232c1dd46ef4dcd23ee06574721ff3280e73837489ff
[+] Snapshot before execution
[+] Running file
[+] Snapshot after execution
[+] Analyzing changes...

[!] Modified files:

[!] New files:

[!] Deleted files:
/sandbox/home/user/Desktop/Slides/34371_L1a_CourseIntroduction_25.pdf
/sandbox/home/user/Desktop/Slides/34371_L1b_Context_25.pdf
/sandbox/home/user/Desktop/Slides/34371_L5_Transport_Layer_Security_25.pdf
/sandbox/home/user/Desktop/Slides/34371_L6_App_Layer_Sec_MQTTCoAPP_25.pdf
/sandbox/home/user/Desktop/Slides/34371_L7_PhysicalLayerSecurity_Wifi_Bluetooth_25.pdf

[?] No output

[+] Checking filesystem changes with 'docker diff'...
[+] Filesystem change: /sandbox/home/user/Desktop/Slides
```

Figure 10: Detection of malicious file `evil.sh`

Destructive Script Simulation The `evil.sh` script demonstrated destructive behavior by deleting files within the `Desktop` directory. These actions were successfully captured in the *deleted files* output (see Figure 10), where multiple files were listed as removed. This validates the system's ability to detect data loss or sabotage through filesystem comparison.

6.3.2 Ransomware Simulation

```
[+] Executable detected: ../alpine-rootfs/home/user/Documents/ransomware.sh
[+] Launching sandbox for: /mnt/c/Users/Lauri/Desktop/Python/Anitvirus/alpine-rootfs/home/user/Documents/ransomware.sh
[+] Building sandbox image...
[+] Starting isolated container...
[+] Container ID: 116c5e295bd7ed4bec591e67c6a1339a8afd86ff0cdd12161ce85e6b486ae1b3
[+] Snapshot before execution
[+] Running file
[+] Snapshot after execution
[+] Analyzing changes...

[!] Modified files:
/sandbox/home/user/Documents/SigintOS_apr_fix.txt
/sandbox/home/user/Documents/arch_linux_qtile_setup.txt
/sandbox/home/user/Documents/communication_tech_lora_probes&beacon_tables.txt
/sandbox/home/user/Documents/vim_cheatsheet.txt
/sandbox/home/user/Documents/windows_storage_clean.txt

[!] New files:

[!] Deleted files:

[!] Malware output:
mv: cannot stat 'report.txt.enc': No such file or directory

[+] Checking filesystem changes with 'docker diff'...
[+] Filesystem change: /sandbox/home/user/Documents/SigintOS_apr_fix.txt
[+] Filesystem change: /sandbox/home/user/Documents/arch_linux_qtile_setup.txt
[+] Filesystem change: /sandbox/home/user/Documents/vim_cheatsheet.txt
[+] Filesystem change: /sandbox/home/user/Documents/communication_tech_lora_probes&beacon_tables.txt
[+] Filesystem change: /sandbox/home/user/Documents/windows_storage_clean.txt
```

Figure 11: Detection of malicious file `ransomware.sh`

Ransomware Simulation The `ransomware.sh` script resulted in multiple file modifications within the user's `Documents` directory. This behavior was correctly reflected in the

modified files output, where several `.txt` files were detected as altered – As depicted in Figure 11. This confirms that the hash-based snapshot mechanism is effective in identifying content changes within monitored directories, which is characteristic of ransomware activity.

6.3.3 Agent Installation Simulation

```
[+] Executable detected: ../alpine-rootfs/home/user/Downloads/agent_installer.sh
[+] Launching sandbox for: /mnt/c/Users/lauri/Desktop/Python/Anitvirus/alpine-rootfs/home/user/Downloads/agent_installer.sh
[+] Building sandbox image...
[+] Starting isolated container...
[+] Container ID: 0913b1ae5e9e102a97f4f9aba005c76671bfa60c5c351706925fa0dc7cb3100f
[+] Snapshot before execution
[+] Running file
[+] Snapshot after execution
[+] Analyzing changes...

[!] Modified files:

[!] New files:

[!] Deleted files:

[!] Malware output:

[*] Installing Python agent...
Starting agent
start-stop-daemon: unable to stat /usr/bin/python3 (No such file or directory)

[+] Checking filesystem changes with 'docker diff'...
[!] Suspicious file type detected: /opt/agent/agent.py
[!] Suspicious change detected: /etc/init.d/agent

[X] Suspicious filesystem changes detected
```

Figure 12: Detection of malicious file `agent.sh`

Agent Installation Simulation The `agent_installer.sh` script exhibited more advanced behavior by attempting to install a persistent agent. Unlike the previous cases, no changes were detected within the monitored `/sandbox/home` directory using hash-based snapshots (see Figure 12). However, Docker’s `diff` functionality revealed significant system-level modifications, including:

- Creation of files in `/opt/agent`
- Addition of a persistence mechanism in `/etc/init.d`

This demonstrates a key limitation of localized snapshotting: changes occurring outside the monitored directory may go undetected. In contrast, `docker diff` provides a comprehensive view of all filesystem changes within the container, making it particularly effective for identifying persistence mechanisms and system-level modifications.

6.4 Testing and Calibrating the Verdict

6.4.1 Traffic Verdict Testing

Monitoring the network activity of a potentially malicious file is important because many malware types rely on external communication during execution. Downloaders and installer scripts often create outbound connections to retrieve additional payloads, communicate with command-and-control infrastructure, or verify internet connectivity. Observing these connections can therefore provide useful behavioral indicators, even when no visible filesystem changes are detected.

A test was performed using the script `mixed-connections.sh` from Appendix ???. The script performs three different outbound connection attempts using `curl`: a standard HTTP header request to a highly reported IP address from AbuseIPDB, a download attempt from a local server, and a header request to a known Tor node. The purpose of the test was to evaluate whether the traffic monitoring component could detect and classify suspicious network-related behavior during sandbox execution.

```
[+] Network connections observed:
tcp -> 192.42.116.15:80 (S, hits=1, out=1, in=0, out payload=0 pkts/0B, in payload=0 pkts/0B)
tcp -> 192.168.0.33:8000 (S, hits=1, out=1, in=0, out payload=0 pkts/0B, in payload=0 pkts/0B)
tcp -> 182.95.109.138:80 (S, hits=1, out=1, in=0, out payload=0 pkts/0B, in payload=0 pkts/0B)
[+] Checking filesystem changes with 'docker diff'...

[!] Modified files:
None

[!] New files:
None

[!] Deleted files:
None

[!] Sample exit codes:
/home/user/Documents/mixed_connections.sh: 28

[+] Traffic verdict:
182.95.109.138 (score=100, reports=1339, country=India, isTor=False)
192.168.0.33 (score=0, reports=9, country=None, isTor=False)
192.42.116.15 (score=100, reports=454, country=Netherlands, isTor=True)

Verdict:
Label: low-confidence suspicious
Score: 10
Reasons:
- network-related behavior observed
[X] Suspicious filesystem changes detected
```

Figure 13: Testing the traffic verdict

Figure 13 shows the output generated during execution of the `mixed-connections.sh` test script. The monitoring component successfully detected multiple outbound TCP connection attempts, including connections to a Tor node and a highly reported IP address from AbuseIPDB. For each observed IP address, the system queries the AbuseIPDB API to retrieve metadata such as abuse reports, country of origin, Tor usage, and an associated abuse confidence score. In the current prototype, this information is presented as part of a separate traffic verdict. Future versions of the verdict logic are intended to incorporate these scores directly into the final malware classification in order to strengthen detection of network-related malicious behavior.

6.4.2 Agent Installation with Traffic Verdict

In order to evaluate the extended monitoring functionality, the agent installation scenario was repeated with the traffic verdict component enabled. The purpose of this test was to observe whether the system could successfully capture and present network-related behavior alongside the existing filesystem-based analysis.

```
[+] Network connections observed:
tcp -> 192.168.65.7:53 (DATA, hits=12, out=6, in=6, out payload=0 pkts/0B, in payload=0 pkts/0B)
tcp -> 172.66.152.176:80 (. , F., P., S, S., hits=168713, out=80238, in=88475, out payload=19 pkts/7190B, in payload=88452 pkts/119921985B)
tcp -> 138.68.95.15:8000 (S, S., hits=2, out=1, in=1, out payload=0 pkts/0B, in payload=0 pkts/0B)
[+] Checking filesystem changes with 'docker diff'...

[!] Modified files:
None

[!] New files:
None

[!] Deleted files:
None

[!] Sample exit codes:
/home/user/Desktop/agent_installer.sh: 0

[+] Traffic verdict:
138.68.95.15 (score=0, reports=0, country=Germany, isTor=False)
172.66.152.176 (score=0, reports=38, country=United States of America, isTor=False)
192.168.65.7 (score=0, reports=0, country=None, isTor=False)

[+] Filesystem changes from docker diff:
/etc/python3/debian_config
/etc/systemd/system/agent.service
/etc/mime.types
/etc/python3.10/sitecustomize.py
/etc/ld.so.cache
/etc/inputrc
/opt/agent/agent.py
/var/lib/python/python3.10_installed
/var/cache/ldconfig/aux-cache

Verdict:
Label: likely malicious
Score: 50
Reasons:
- modified or created executable content
- touched sensitive system paths
- installed package(s): libmpdec3:amd64, libmpdec3:amd64., libpython3-stdlib:amd64, libpython3-stdlib:amd64., libpython3.10-minimal:amd64, libpython3.10-minimal:amd64., libpython3.10-stdlib:amd64, libpython3.10-stdlib:amd64., libreadline8:amd64, libreadline8:amd64., libsqlite3-0:amd64, libsqlite3-0:amd64., media-types, media-types., python3, python3-certifi, python3-certifi., python3-chardet, python3-chardet., python3-idna, python3-idna., python3-minimal, python3-minimal., python3-pkg-resources, python3-pkg-resources., python3-requests, python3-requests., python3-six, python3-six., python3-urllib3, python3-urllib3., python3., python3.10, python3.10-minimal, python3.10-minimal., python3.10., readline-common, readline-common.
- network-related behavior observed
[X] Suspicious filesystem changes detected
```

Figure 14: Agent Installer with Traffic Verdict

The traffic output also confirms that the sandbox successfully connected to the externally hosted server at 138.68.95.15, from which the agent payload was retrieved. Furthermore, the resulting filesystem changes show that the agent was successfully installed and moved into the target system paths inside the sandbox environment.

6.5 Testing Full Malware Program: RedTail

Malware samples collected from MalwareBazaar are often incomplete and may only contain setup scripts or payload installers rather than the full malware executable. Because of this, a Cowrie honeypot was deployed with weak SSH credentials in order to attract automated bot scanners and collect real-world malware samples. During the collection period, the honeypot captured a sample identified as RedTail.

RedTail is a Linux-based cryptominer that targets exposed systems and installs persistent mining payloads through services and cron jobs. Before the run shown in Figure 9, the verdict logic had already been updated to recognize RedTail-like behavior, especially dropped payloads, persistence in sensitive system paths, and multi-architecture miner artifacts. The purpose of this test was therefore to verify that the updated verdict correctly classified a realistic malware sample in the Docker-based sandbox.

```
===== Running /home/user/Documents/setup.sh =====
[+] Checking filesystem changes with 'docker diff'...
[!] Modified files:
None
[!] New files:
None
[!] Deleted files:
None
[!] Sample exit codes:
/home/user/Documents/clean.sh: 0
/home/user/Documents/setup.sh: 0
[+] Filesystem changes from docker diff:
/.0wrqQUQdgzYYoHsQw9X0C5vb
/home/user/Documents/redtail.x86_64
/home/user/Documents/redtail.arm7
/home/user/Documents/redtail.arm8
/home/user/Documents/redtail.i686
/etc/hosts.deny
/etc/systemd/system/your-redtail.service
/etc/hosts.allow
/etc/anacrontab
/etc/cron.d/e2scrub_all
/etc/cron.daily/apt-compat
/etc/cron.daily/dpkg
Verdict:
Label: likely malicious
Score: 110
Suspected behavior: cryptominer
Reasons:
- touched sensitive system paths
- cryptominer-associated payload artifact observed
- cryptominer payload installed with persistence
- multiple architecture-specific miner payloads observed
[X] Suspicious filesystem changes detected
```

Figure 15: RedTail Cryptominer collected on HoneyPot

Based on the observed activity, the verdict system classified the sample as likely malicious with a high behavior score. The detection was primarily triggered by the creation of executable payloads, modifications to sensitive system paths, and the installation of persistence mechanisms commonly associated with cryptomining malware.

6.6 Results of the Previous Rule-Based Approach

The early testing phase showed that the implemented monitoring pipeline was able to capture relevant behavioral artifacts from executed samples. Across the different test scenarios, the system successfully observed file modifications, file deletion activity, creation of new executables, network-related behavior, and persistence-related actions. These results confirmed that the Docker-based sandbox and the surrounding Python control logic were functioning as intended and that the collected execution data was sufficiently detailed to support automated assessment. In this sense, the rule-based verdict approach was useful as an initial validation step, since it demonstrated that suspicious behavior could be detected and translated into a preliminary classification.

6.7 Limitations of the Previous Rule-Based Verdict

Although the previous approach was effective for detecting clearly defined malicious actions, the testing also revealed important limitations. The hardcoded verdict depended on manually selected thresholds, explicit score assignments, and predefined combinations of behaviors. This made the decision logic difficult to generalize beyond the specific scenarios used during development. Behaviors that were individually benign could become suspicious in combination, while some malicious patterns were difficult to represent through fixed rules alone. As a result, the verdict logic became increasingly brittle as more edge cases and behavioral variations were considered, making further refinement more complex and less scalable.

6.8 Why the System Transitioned to an AI Agent

The earlier tests were therefore not wasted effort, but instead served as an important foundation for the next design step. They validated that the system could reliably execute suspicious files, collect meaningful behavioral artifacts, and identify the types of observations that should influence a final verdict. At the same time, they showed that converting these observations into a robust hardcoded decision model was difficult. This motivated the transition toward an AI-agent-based assessment approach, where the collected execution report is analyzed contextually rather than only through fixed scoring rules. The AI agent was therefore introduced not to replace the earlier work, but to build on it by improving the system's ability to interpret combined behavioral evidence in a more flexible and scalable way.

Following the previous test phase, the collected output was refined into a structured JSON format containing the behavioral information most relevant to the final assessment. This JSON report serves as the input basis for the AI agent and gathers the observed execution artifacts in a form that can be processed more consistently. In addition, an AI persona-based approach was introduced by initializing the agent with a predefined persona described in a YAML file. This persona defines the intended analytical role and helps guide the style and focus of the generated verdict. Strictly speaking, neither JSON nor YAML is strictly required, since the agent is capable of reasoning over unstructured text alone.

However, the use of structured formats improves modularity and maintainability, and also makes the system easier to extend in the future, for example toward visualization in a web-based user interface.

6.9 AI Agent-Based Verdict Testing

To evaluate the AI-based verdict, a set of tests similar to the earlier scenarios was conducted using the same input cases from the `Tests` directory. The purpose was to examine whether an AI agent could produce a consistent severity assessment when presented with the structured execution report instead of relying on hardcoded verdict logic. Four locally hosted models were tested through Ollama: `llama3.1:latest`, `mistral:latest`, `qwen2.5-coder:7b`, and `mistral-nemo:12b`. For each test case, the generated verdict was compared in terms of severity level and numerical risk score.

Table 5 shows the resulting severity assessments. Overall, the outputs were relatively consistent for the clearly suspicious cases, although the models differed in how aggressively they assigned the highest severity. Differences were also visible between the four reported models in both risk score calibration and runtime behavior, which made the comparison useful not only for classification quality but also for practical performance in repeated local testing.

6.9.1 Selected models for testing

The selected model set consisted of `llama3.1:latest`, `mistral:latest`, `qwen2.5-coder:7b`, and `mistral-nemo:12b`. These models were chosen to provide a mix of general-purpose instruction models and a code-oriented model, while still remaining practical to run locally through Ollama.

The evaluation was performed on five behavioral test cases from the `Tests` directory: `agent_test`, `cryptominer_test`, `legitimate_test`, `stealth_test`, and `traffic_test`. Together, these cases were used to check how the models reacted to clearly suspicious behavior, ambiguous or stealth-oriented behavior, ordinary benign execution, and samples with notable network-related activity. On that basis, the following comparison table summarizes the severity labels and risk scores assigned by each model.

6.9.2 Model comparison

Test case	llama3.1	mistral	qwen2.5-coder	mistral-nemo
<code>agent_test</code>	High (80)	High (85)	Medium (50)	High (75)
<code>cryptominer_test</code>	Critical (95)	Medium (40)	High (85)	Medium (60)
<code>legitimate_test</code>	Low (20)	Low (10)	Low (20)	Low (10)
<code>stealth_test</code>	High (80)	Medium (40)	Medium (30)	Medium (60)
<code>traffic_test</code>	High (80)	Medium (50)	Low (20)	Low (10)

Table 5: Comparison of severity levels and risk scores produced by the tested Ollama models.

The comparison shows that `llama3.1:latest` produced the most consistently high-severity assessments across the suspicious cases, while also keeping the legitimate case in

the *Low* range. `mistral:latest` and `mistral-nemo:12b` were generally more conservative in their scoring, whereas `qwen2.5-coder:7b` often gave lower scores for stealth and traffic-oriented behavior but still identified the cryptominer scenario as *High*.

6.9.3 Runtime comparison

Model	Total time (s)	Avg/test (s)	Tests
<code>llama3.1:latest</code>	19	3.8	5
<code>mistral:latest</code>	21	4.2	5
<code>qwen2.5-coder:7b</code>	20	4.0	5
<code>mistral-nemo:12b</code>	79	15.8	5

Table 6: Runtime comparison for the reported models across the five AI-verdict test cases.

In practical terms, the runtime results show that `llama3.1:latest`, `mistral:latest`, and `qwen2.5-coder:7b` were all within a narrow range of roughly four seconds per test case. By contrast, `mistral-nemo:12b` was significantly slower, averaging 15.8 seconds per test, which makes it notably less efficient for repeated local evaluation despite producing usable results.

6.9.4 Reference Severity for the Test Cases

To interpret the model outputs beyond their mutual agreement, it is useful to establish a reference expectation for each test case, derived from the nature of the behavioral artifacts contained in the corresponding execution report. These expectations are analyst-derived rather than externally validated labels, but they provide a consistent baseline against which the calibration of each model can be discussed. Table 7 summarizes these expectations together with the dominant indicator driving each one.

Test case	Expected severity	Dominant indicator
<code>legitimate_test</code>	Low	<code>apt</code> package updates from Canonical/Ubuntu repositories, updates are however triggered via a script
<code>agent_test</code>	Medium-High	<code>systemd</code> persistence and unknown agent service, downloader is from non-flagged IP
<code>cryptominer_test</code>	Critical	Multi-architecture payloads with <code>systemd</code> and <code>cron</code> persistence
<code>stealth_test</code>	High	<code>cron</code> persistence with hidden, disguised worker artifacts tailored to trick the Agent
<code>traffic_test</code>	High	Outbound contact with abuse-flagged IPs and a Tor exit node, highly suspicious connections but no downloads

Table 7: Reference severity expectations used to assess model calibration, with the primary behavioral indicator motivating each expectation.

6.9.5 Discussion of Verdict Quality

Measured against these expectations, the four models diverged most clearly on the two cases that the rule-based predecessor was least equipped to handle: the network-reputation case (`traffic_test`) and the stealth-oriented persistence case (`stealth_test`).

`llama3.1:latest` produced the closest overall alignment with the reference expectations. It was the only model to escalate `traffic_test` to *High*, correctly identifying both the abuse-flagged destination and the Tor exit node, and it was also the only model to rate `stealth_test` as *High*. It assigned the cryptominer sample the highest severity of any model (*Critical*) while still keeping the legitimate update script in the *Low* range. Its principal weakness was not severity calibration but classification labelling: it repeatedly applied the label *credential stealer* even to the cryptominer and stealth samples, where the evidence does not support that specific category.

`mistral:latest` produced well-formed output and reasonable labels, but was systematically conservative. It under-scored both the cryptominer sample (*Medium*, 40) and the stealth sample (*Medium*, 40), and rated the network case only *Medium* (50) despite explicitly noting the Tor exit node in its own evidence list. This indicates that the model recognized the relevant indicators but did not escalate the resulting severity accordingly.

`qwen2.5-coder:7b` correctly identified the cryptominer sample as *High*, but produced the most pronounced under-call on the network case, rating `traffic_test` as *Low* (20) and stating that no connections to known-malicious infrastructure were present. This conflicts with the reputation data in the report and represents a false negative on genuinely flagged infrastructure.

`mistral-nemo:12b` produced the weakest result on the network case, classifying `traffic_test` as benign (*Low*, 10) on the grounds that no payload was exchanged and no filesystem changes occurred, while disregarding the reputation verdicts entirely. It also under-rated the cryptominer sample. Combined with its runtime (discussed below), this makes it the least favorable trade-off in the set.

6.9.6 Cross-Model Observations

Two patterns emerge from the comparison that are more informative than any single model's score.

First, agreement was high on the unambiguous cases and low on the ambiguous ones. All four models rated `legitimate_test` as *Low*, and all four treated the multi-architecture cryptominer payloads as suspicious, differing only in how aggressively they scored them. Divergence was concentrated in the stealth and network cases, which is precisely the region where contextual interpretation was expected to add value over fixed rules.

Second, the network case acted as the primary discriminator between the models. Three of the four did not translate the structured reputation data (`traffic_verdict`, including abuse scores and the `isTor` flag) into an elevated severity, and two produced explicit false negatives. The common cause was an over-reliance on filesystem activity: because `traffic_test` contained no file changes, the models that under-scored it appear to have treated the absence of filesystem modification as evidence of benign behavior, rather than weighting the network reputation signal independently. This is an interpretation gap rather than a knowledge gap, and it is notable because it is a failure mode a hardcoded rule keyed on `score` or `isTor` would not exhibit. The AI-based approach therefore trades the brittleness of fixed rules for a degree of interpretation variance, which has direct implications for persona design: signals that must never be ignored should be named explicitly in the persona and bound to an escalation rule, rather than left to be inferred.

6.9.7 Severity Calibration versus Classification Accuracy

A further observation is that the numeric `risk_score` and severity label were more reliable across models than the free-text `classification` field. Several models arrived at a defensible severity while attaching an inaccurate category, with *credential stealer* applied to samples whose behavior is more consistent with mining or generic persistence. For a triage pipeline this is an acceptable ordering of strengths, since the severity drives prioritization while the classification is advisory, but it suggests that downstream consumers should treat the classification label as a hint rather than an authoritative determination.

6.9.8 Runtime and Practical Trade-offs

The runtime results in Table 6 reinforce the quality findings. The three 7–8B models clustered around four seconds per case, whereas `mistral-nemo:12b` required roughly four times as long at 15.8 seconds per case. Crucially, this additional cost did not translate into better verdicts: `mistral-nemo:12b` was both the slowest model and one of the two that failed the network case outright. Within this set, larger parameter count therefore did not correlate with higher assessment quality, and the practical sweet spot for repeated local evaluation lay firmly in the 7–8B tier.

6.9.9 Model Selection

Taking calibration, error direction, and runtime together, `llama3.1:latest` is the most suitable model for this pipeline. It was the only model to handle all five cases in the expected direction, it produced the fewest dangerous false negatives, and it was tied for the fastest runtime. For a security-triage setting in which a missed detection is more costly than an over-cautious flag, its tendency to score slightly high on ambiguous cases is the preferable failure direction, in contrast to the under-scoring observed in the other three models on the stealth and network cases.

6.9.10 Threats to Validity

Several limitations should be acknowledged. The reference severities in Table 7 are analyst-derived and not validated against an external ground-truth corpus, so they describe expected rather than authoritative labels. The evaluation covers five test cases, which is sufficient to expose behavioral differences but too small to support statistical claims about accuracy. Each case was assessed in a single run per model; because local generation is non-deterministic unless the temperature is fixed, repeated runs may yield some variation in the exact scores, and reporting per-case variance would strengthen future comparisons. Finally, the outputs depend on a shared persona definition, so the observed differences reflect the combination of model and persona rather than the model in isolation; a revised persona that names the reputation signal explicitly would be expected to shift the network-case results for the conservative models in particular.

7 Discussion

The original idea was to emphasize scalable malware scanning through infrastructure built around Docker and Kubernetes. This would have made it possible to distribute analysis workloads across multiple environments and study the scalability of sandboxed execution. During development, however, the project direction changed because the Kubernetes-related part introduced major implementation difficulties. In particular, an unexpected runtime change from Docker to containerd made it difficult to establish a natural and reliable interface between the Docker-based sandboxing block and the Kubernetes block.

As a result, the project scope was narrowed rather than fundamentally changed. The core objective remained behavior-based malware detection, but the broader infrastructure and orchestration focus was reduced so that the implementation could concentrate on the behavioral analysis logic itself. This made it possible to prioritize isolated execution, observation of runtime effects, and verdict generation, which were more central to the main goal of identifying suspicious actions performed by unknown files.

7.1 Scope Refinement

The resulting prototype therefore prioritizes controlled execution and behavioral evidence collection instead of orchestration complexity. Docker remains central because it provides a practical and lightweight sandboxing mechanism. Kubernetes, on the other hand, is no longer part of the implemented core system, even though it remains relevant as part of the original design motivation and as a possible future extension.

7.2 Tradeoffs

This refinement introduces a clear tradeoff. The system is now less focused on large-scale deployment and distributed scanning, but more focused on the quality and relevance of the analysis performed on each suspicious sample. For the current project stage, this is considered a reasonable tradeoff, since the ability to observe and interpret malicious behavior is more important than demonstrating infrastructure scaling on top of an incomplete analysis core.

7.3 Limitations

The present implementation still has several limitations. The behavioral verdict is based on heuristics and a limited set of monitored effects. The test scenarios are also relatively narrow compared to real-world malware diversity. Furthermore, the prototype does not yet include a broad performance evaluation or large-scale deployment study.

A further limitation concerns access to representative real-world malware samples. In practice, it is difficult to obtain malware that is both safe to handle and complete enough for realistic testing. Public sources such as MalwareBazaar often contain partial payloads,

setup scripts, or incomplete archives, which means that some samples do not execute fully inside the sandbox. This limits the realism of the evaluation and makes it harder to assess the system against complete real-world malware behavior.

7.4 Future Perspective

Even though Kubernetes was removed from the implementation focus, the idea of distributed sandboxed analysis remains relevant. If the behavior-based core is matured and validated further, orchestration technologies could later be reintroduced as a way to scale the system. In that sense, the infrastructure-oriented idea is not discarded completely, but postponed until the analysis logic itself is sufficiently strong.

8 Conclusion

This midterm report presents the current state of a behavior-based malware-analysis prototype built around isolated execution in Docker containers. The implemented system is capable of scanning target paths, executing suspicious files in a sandboxed environment, and identifying relevant behavioral changes such as file creation, deletion, and modification.

During the project, the scope was narrowed from a broader infrastructure and scalability perspective to a more focused implementation of the malware-analysis core. This refinement led to a clearer and more practical prototype, where the main emphasis is placed on isolated execution, observation of runtime behavior, and verdict generation rather than on large-scale orchestration.

The current results indicate that the approach is technically feasible and relevant within the project scope. The next phase of the work is, as discussed in Section ??, to further refine the verdict logic, integrate the traffic-based scoring component, and perform additional validation using representative malware samples. In its current form, the system provides a foundation for further refinement and evaluation during the final phase of the project.

Use of Artificial Intelligence Tools

The following AI tools were used during the development of the project and preparation of this report:

Provider	Model	Purpose
Claude	Sonnet	Generation of architecture diagrams, figure brainstorming, and report structure discussions.
Claude	Opus 4.8	Development and refinement of the Ollama wrapper used by the AI analysis agent.
OpenAI	GPT-5.5	Technical discussions, report refinement, proof-reading, Docker theory discussions, and general project support.
OpenAI	Codex 4.8	Development of helper functions, code suggestions, debugging assistance, and support for the legacy rule-based verdict implementation.

Table 8: Artificial intelligence tools used during the project.

All design decisions, implementation work, testing, analysis, and final report content remain the responsibility of the author.

References

- [1] J. Nickoloff, *Docker in Action*. Manning Publications, Company, 2016.
- [2] J. Nickoloff and S. Kuenzli, *Docker in Action, Second Edition*. Manning, 2019.
- [3] A.-T. Institute, “Statistics: New malware,” 2024. Accessed June 2026.
- [4] R. Sihwail, K. Omar, and K. A. Zainol Ariffin, “A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis,” *International Journal on Advanced Science, Engineering and Information Technology*, vol. 8, no. 4-2, p. 1671, 2018.
- [5] A. Damodaran, A. Visaggio, Corrado, D. Troia, Fabio, M. Stamp, and H. Austin, Thomas, “A comparison of static, dynamic, and hybrid analysis for malware detection,” *Journal of Computer Virology and Hacking Techniques*, vol. 13, no. 1, pp. 1–12, 2017.
- [6] S. Nimmagadda, “Linux namespaces and cgroups as os primitives for lightweight virtualization: Architecture, isolation mechanisms, and performance evaluation,” *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, vol. 9, pp. 811–822, 08 2018.
- [7] Tejun Heo, “Control groups v2 documentation,” 2023.
- [8] Docker Inc., *Docker Documentation*, 2026. Accessed: 2026-03-27.
- [9] R. Shu, X. Gu, and W. Enck, “A study of security vulnerabilities on docker hub,” in *Proceedings of CODASPY*, 2017.
- [10] M. Sikorski and A. Honig, *Practical malware analysis: The hands-on guide to dissecting malicious software*. No Starch Press, 2012.
- [11] J. Saxe and K. Berlin, “Deep neural networks for malware detection,” *Journal in Computer Virology and Hacking Techniques*, vol. 11, no. 2, pp. 59–79, 2015.
- [12] G. E. Dahl, N. Jaitly, and R. Salakhutdinov, “Multi-task neural networks for qsar predictions,” *arXiv preprint arXiv:1406.1231*, 2014.
- [13] G. Stringhini, C. Kruegel, and G. Vigna, “Detecting malicious flash advertisements,” in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pp. 269–284, IEEE, 2010.
- [14] C. Kolbitsch, P. M. Comparetti, C. Kruegel, E. Kirda, X. Y. Zhou, and X. Wang, “Effective and efficient malware detection at the end host,” in *USENIX security symposium*, vol. 1, p. 1, 2009.
- [15] H. Carvey, *Windows Forensic Analysis: DVD Toolkit*. Syngress, 2007.

- [16] W. Huang, J. W. Stokes, M. Marinelli, T. Brown, L. Invernizzi, N. O. Tippenhauer, others, and J. Saxe, “Adversarial deep learning for over-the-air spectrum poisoning attacks against convolutional radio classifiers,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 4, pp. 713–729, 2016.
- [17] M. Wooldridge, *An introduction to multiagent systems*. John Wiley & Sons, 2009.
- [18] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, others, and D. Zhou, “Emergent abilities of large language models,” *arXiv preprint arXiv:2206.07682*, 2022.
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, others, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- [20] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?”: Explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.
- [21] G. Demetrio, A. Valenza, G. Costa, and G. Lagorio, “Intriguing properties of neural networks for malware analysis,” in *USENIX Security Symposium*, 2021.
- [22] B. Goodman and S. Flaxman, “European union regulations on algorithmic decision-making and a “right to explanation”,” *AI Magazine*, vol. 38, no. 3, pp. 50–57, 2017.
- [23] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.
- [24] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems*, pp. 6402–6413, 2017.
- [25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichien, F. Xia, others, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [26] Z.-H. Zhou, *Ensemble methods: Foundations and algorithms*. CRC press, 2012.